Diagnostic Quick-reference Booklet (SGI™ 3000 Family)

Document Number 108-0290-001

CONTRIBUTORS

Written by Stacey Delcore
Edited by Allison Gosbin
Production by Rhonda Kunsman
Engineering Contributions by Lisa Steinmetz, Karen Beighley, Louis Grannis III, Jon Palm, Todd Andringa, Nelson Crumbaker, Joe Ivanauskas, Darin Oster, Paul Ebben, Judy Young, Jeff Keopp, Jason Godfrey, Gary Davidson, Sanjay Gupta, Roberto Romano, and Gregoire Banderet

INFORMATION CLASSIFICATION

LIMITED RIGHTS LEGEND

TRADEMARKS

Contents

Figures

Tables

Record of Revision

Version	Description
001	September 2000 Original printing
	November 2000 Updated for IRIX 6.5.10

About This Booklet

This publication documents the SGI 3000 family diagnostics that are available to field engineers.

Notational Conventions

Convention	Meaning
bold	Bold typeface denotes important information to which the reader should pay extra attention
`screen display`	Fixed-space font denotes literal items such as commands, files, routines, path names, output, messages, and programming language structures
variable	Italic typeface denotes variable entries and words or concepts that are being defined
`user input`	Bold, fixed-space font denotes literal items that the user enters in interactive sessions
...	Ellipses indicate that one or more elements have been removed and/or that the preceding element can be repeated
[]	Square brackets enclose optional items
<>	Pointed brackets enclose required portions of an optional item
|	Dividers are used to separate possible values of an item

Related Documentation

Refer to the following documents for more information about the SGI 3000 family diagnostic suite:

- *Scan Tools*, publication number 108-0263-001
- *Power-on Diagnostics and Discovery*, publication number 108-xxxx-001
- *sMDK-based Field Diagnostics*, publication number 108-0264-001
- *Online Diagnostics*, publication number 108-0286-002

Online documentation is also available at the Service Publications and Training Web site: `http://servinfo.csd.sgi.com`

New Releases

Field engineers can receive new releases of SGI software by obtaining the corresponding release of the Internal Support Tools (IST) CD or by referring to the System Support Tools Web site: `http://ist.csd.sgi.com`.

L1, L2, and L3 system controller software and boundary scan software updates are also available to field engineers at the following location: `dist.engr:/test3/field_diags/linux/irix65<release_number>`.

Offline (sMDK) and online diagnostic updates are available to field engineers at the following location: `dist.engr:test/CD/6.5.<release_number>/latest/field_diags`.

External users can obtain new releases of L1 and L2 system controller software from the corresponding IRIX CD or from one of the following locations:

- `dist.engr:/test/CD/6.5.<release_number>/latest/cd1/dist/eoe_65<release_number>m`
- `dist.engr:/test/CD/6.5.<release_number>/latest/cd2/dist/eoe_65<release_number>f`

Chapter 1

Diagnostic Tests

1.1 Test Categories

Table 1-1 Test Categories

Category	Description
Scan tests	Use JTAG/boundary scan features to verify system interconnections
Power-on diagnostics (POD) and discovery	Test hardware required to boot the system and run as part of the power-up and discovery process
Scalable Micro-diagnostic Kernel (sMDK) diagnostics	Test hardware that is not accessible through the IRIX operating system and that requires full access to the system
Online diagnostics	Run from the IRIX prompt while user operations continue on the system

1.2 Tests by Bricks

Table 1-2 Tests Available by Brick

Brick	Category	Tests
C brick	Scan	`scantest`
		`slit`
	POD	`dirtest`
		`memtest`
	sMDK	`ndir`
		`nmem`
		`scct`
	IRIX	`olmem`
		`olperi`
		`torpedo`

Table 1-2 (continued) Tests Available by Brick

Brick	Category	Tests
R brick	Scan	scantest
		slit
	POD	chklink
		hubsde
		rtrsde
	sMDK	barr
		rtrb
		rtrdmp
		rtrreg
		rtrt
		topology
	IRIX	olrtr
I brick	Scan	scantest
		slit
	POD	dgbrdg
		dgconf
		dgpci
		dgsdma
		dgspio
		dgxbow
		diag_enet
		diag_fibre
	sMDK	io7bt
		xbg
	IRIX	bridgeloc
		olenet
		olpci
		olvst

Table 1-2 (continued) Tests Available by Brick

Brick	Category	Tests
X brick	Scan	scantest
		slit
	POD	dgbrdg
		dgxbow
	sMDK	netkiller
		xbg
	IRIX	olvht
		olvst
P brick	Scan	scantest
		slit
	POD	dgbrdg
		dgpci
		dgxbow
	sMDK	xbg
	IRIX	bridgeloc
		olpci
		olvst
All (stress tests)	sMDK	System_Level_Test
	IRIX	grizzly
		pandora

Chapter 2

Scan Tools

The scan tools use JTAG/boundary scan features to verify system interconnections.

2.1 Running Scan Tools

Warning: **Ensure that the system is offline before you run a scan test.**

1. Install the cables (normal or loopback) necessary for the configuration that you want to test.
2. Install any scannable daughter cards (SPTBs or SDTBs) necessary for the configuration that you want to test.
3. Power up the system.
4. Open a Linux terminal window.
5. Log into the L3 system controller as `sgidiag`.
6. Start the L2 system controller.
7. Connect to the L2 by using the hardware L2 connection or the L2 emulator. Refer to Section 2.1.1, "Using the Hardware L2 Connection," and Section 2.1.2, "Using the L2 Emulator," for more information.
8. Start the appropriate test from the L3 system controller prompt:
   - For the hardware L2 connection—
     `scantest -f` *<file>* `--l2` *<IP address>* `[`*test_options*`]` or
     `slit -f` *<file>* `-i` *<file>* `--l2` *<IP address>* `[`*test_options*`]`
   - For the L2 emulator—
     `scantest -f` *<file>* `[`*test_options*`]` or
     `slit -f` *<file>* `-i` *<file>* `[`*test_options*`]`
9. Interpret the output.

2.1.1 Using the Hardware L2 Connection

1. Connect the L3 to the L2 with an Ethernet connection by using a crossover cable.
2. Enter the command `cat /var/state/dhcp/dhcpd.leases` to obtain the IP address of the L2.
3. Enter the command `/stand/sysco/bin/l2term --l2 <`*IP address*`>`.

   Note: If this does not work, power cycle the L2 by disconnecting and replugging the power cable at the L2. Then recheck the `dhcpd.leases` file for the IP address that correlates to the time stamp of the power cycle.

2.1.2 Using the L2 Emulator

To use the L2 emulator, enter the command `/stand/sysco/bin/l2`.

2.2 Scan Tool Configuration Files

The configuration files are available in the `/stand/sysco/cfg/scan/` directory. Use the following formula to change the `ID` field in a configuration file and select a different brick to test: (*rack_number* * 64) + *slot_number*.

Table 2-1 Scan Tool Configuration Files

File	Description
`c001.cfg`	P1 or pilot C brick with c001 motherboard (no test vectors for `scantest`)
`c00100ni.cfg`	P1 or pilot C brick with c001 motherboard and II and NI loopback cables (no PIMMs)
`c00105.cfg`	P1 or pilot C brick with c001 motherboard and PIMM 1 (no PIMM 0)
`c00105_po.cfg`	P1 or pilot C brick with PIMM only [selects PIMM 1 (nets between processor and SRAM)]
`c00150.cfg`	P1 or pilot C brick with c001 motherboard and PIMM 0 (no PIMM 1)
`c00150_po.cfg`	P1 or pilot C brick with PIMM only [selects PIMM 0 (nets between processor and SRAM)]
`c00155.cfg`	P1 or pilot C brick with c001 motherboard, PIMM 0, and PIMM 1
`c00155ni.cfg`	P1 or pilot C brick with c001 motherboard, PIMM 0, PIMM 1 and II and NI loopback cables

Table 2-1 (continued) Scan Tool Configuration Files

File	Description
c001dimm0.cfg	P1 or pilot C brick with c001 motherboard SDTB in DIMM 0
c001dimm1.cfg	P1 or pilot C brick with c001 motherboard SDTB in DIMM 1
c001dimm2.cfg	P1 or pilot C brick with c001 motherboard SDTB in DIMM 2
c001dimm3.cfg	P1 or pilot C brick with c001 motherboard SDTB in DIMM 3
c001dimm4.cfg	P1 or pilot C brick with c001 motherboard SDTB in DIMM 4
c001dimm5.cfg	P1 or pilot C brick with c001 motherboard SDTB in DIMM 5
c001dimm6.cfg	P1 or pilot C brick with c001 motherboard SDTB in DIMM 6
c001dimm7.cfg	P1 or pilot C brick with c001 motherboard SDTB in DIMM 7
c001dimm04.cfg	P1 or pilot C brick with c001 motherboard SDTBs in DIMM 0 and DIMM 4
c001dimm15.cfg	P1 or pilot C brick with c001 motherboard SDTBs in DIMM 1 and DIMM 5
c001dimm26.cfg	P1 or pilot C brick with c001 motherboard SDTBs in DIMM 2 and DIMM 6
c001dimm37.cfg	P1 or pilot C brick with c001 motherboard SDTBs in DIMM 3 and DIMM 7
i001.cfg	P1 I brick (no test vectors for scantest)
i001_a2b.cfg	P1 I brick with cable between ports A and B
i001_a.cfg	P1 I brick with loopback cable on port A
i001_b.cfg	P1 I brick with loopback cable on port B
i001_pb1.cfg	P1 I brick with SPTBs in bus 1 (all slots)
i001_pb2.cfg	P1 I brick with SPTBs in bus 2 (all slots)
i002.cfg	Pilot I brick (no test vectors for scantest)
i002_a2b.cfg	Pilot I brick with cable between ports A and B
i002_a.cfg	Pilot I brick with loopback cable on port A
i002_b.cfg	Pilot I brick with loopback cable on port B
i002_pb1.cfg	Pilot I brick with SPTBs in bus 1 (all slots)
i002_pb2.cfg	Pilot I brick with SPTBs in bus 2 (all slots)
p001.cfg	P1 or pilot P brick
p001_a2b.cfg	P1 or pilot P brick with cable between port A and B
p001_a.cfg	P1 or pilot P brick with loopback cable on port A
p001_b.cfg	P1 or pilot P brick with loopback cable on port B
p001_pb1.cfg	P1 or pilot P brick with SPTBs in bus 1 (all slots)

Table 2-1 (continued) Scan Tool Configuration Files

File	Description
p001_pb2.cfg	P1 or pilot P brick with SPTBs in bus 2 (all slots)
p001_pb3.cfg	P1 or pilot P brick with SPTBs in bus 3 (all slots)
p001_pb4.cfg	P1 or pilot P brick with SPTBs in bus 4 (all slots)
p001_pb5.cfg	P1 or pilot P brick with SPTBs in bus 5 (all slots)
p001_pb6.cfg	P1 or pilot P brick with SPTBs in bus 6 (all slots)
r001.cfg	P1 or pilot R brick with cables that connect ports H and A, ports B and C, ports D and E, and ports F and G
r001_bc.cfg	P1 or pilot R brick with loopback cables on ports B and C and cables that connect ports A and H, ports D and E, and ports F and G
r001_de.cfg	P1 or pilot R brick with loopback cables on ports D and E and cables that connect ports A and H, ports B and C, and ports F and G
r001_fg.cfg	P1 or pilot R brick with loopback cables on ports F and G and cables that connect ports A and H, ports B and C, and ports D and E
r001_ha.cfg	P1 or pilot R brick with loopback cables on ports H and A and cables that connect ports B and C, ports D and E, and ports F and G
x001_89.cfg	P1 X brick with cable between ports A and B and Crosstown boards in slots 8 and 9
x001_a2b.cfg	P1 X brick with cable between ports A and B
x001_a.cfg	P1 X brick with loopback cable on port A
x001_all.cfg	P1 X brick with cable between ports A and B and Crosstown boards in slots 8, 9, C, and D
x001_b.cfg	P1 X brick with loopback cable on port B
x001_cd.cfg	P1 X brick with cable between ports A and B and Crosstown boards in slots C and D

2.3 scantest (Boundary Scan Interconnect Test)

The `scantest` manipulates the boundary scan registers in various ASICs throughout the system to verify system interconnections. It also applies a set of test vectors to detect physical connection defects (for example, stuck-at faults and shorts).

`$SCAN_PATH_BIN/scantest [-h] <-f <$SCAN_PATH_CONFIG/`*file*`>> [-d <`*file*`>] [-e <`*num*`>] [-p <`*num*`>] [-t <`*test*`>] [-m <`*mode*`>] [-l <`*level*`>] [-C] [-s <`*mode*`>] [-v]`

Table 2-2 scantest Command-line Options

Option	Description
`-h`	Displays all available command-line options and exits
`-f <`*file*`>`	Uses the specified configuration file (Refer to Section 2.2, "Scan Tool Configuration Files," for valid `scantest` configuration files; default: `scantest.cfg`)
`-d <`*file*`>`	Sends output to the specified file (default: `scantest.dmp`)
`-e <`*num*`>`	Displays only the specified number of errors (`0`–`10000`; `0` = display only pass/fail information; default: `200`)
`-p <`*num*`>`	Performs the specified number of passes (`1`–`10000`; default: `1`)
`-t <`*test*`>`	Runs the specified tests `irp` = instruction register path test `irl` = instruction register length test `ir` = instruction register path and length tests `prp` = bypass register path test `prl` = bypass register length test `pr` = bypass register path and length tests `brp` = boundary register path test `brl` = boundary register length test `br` = boundary register path and length tests `int` = boundary register interconnect test `id` = reads and displays the device ID registers `all` = all tests (Use + to select multiple tests; default: `ir+pr+br+int`)
`-m <`*mode*`>`	Uses the specified mode for the register tests `normal` = normal lengths and minimal pattern(s) (no other option is valid; default) `elen` = extended lengths `epat` = extended pattern set

Table 2-2 (continued) scantest Command-line Options

Option	Description
`-l` *<level>*	Selects the level of the register tests `chn` = chain level (default) `brd` = board level `chip` = chip level
`-C`	Selects an individual SIC to run the test
`-s` *<mode>*	Selects a stepping mode `no` = does not perform test stepping; runs continuously (default) `all` = steps through each pattern; prompts user `err` = steps through the failing pattern; prompts user
`-v`	Selects verbose mode

2.3.1 scantest Error Output Format

```
BOUNDARY REGISTER INTERCONNECT FAILURE
--------------------------------------
❶ Chain and raw position : 000-000601
❷ Expected data          : 0110011010010101
❸ Actual data            : 0000000000000100
❹ Difference data        :   ^^  ^^ ^  ^   ^
❺ Driving node index     : 1111111111110011
❻ Logical description    : [101i01:I:XBG0:PF_PCI_SERR_N]
101i01:I:IOC3:PCI_SERR_L
❼ Physical description: [101i01:I:H7F9:AD21]     101i01:I:J5D4:N2
```

❶ The location where `scantest` detected a miscompare.

❷ The signature that `scantest` expects to receive for the data patterns that it is using.

❸ The actual signature that was sensed for the data patterns.

❹ The difference data marks each pattern in the signature with a ^ symbol where the actual data did not match the expected data.

❺ The node that sent the data.

❻ The logical description of the nodes on the failing net (*<chain and brick>*`:`*<board>*`:`*<chip>*`:`*<pin>*); the node shown in `[]` detected the miscompare. Refer to Figure 2-1 for chain and brick decoding information.

⑰ The physical description of the nodes on the failing net (*<chain and brick>:<board>:<chip>:<pin>*); the node shown in `[]` detected the miscompare. Refer to Figure 2-1 for chain and brick decoding information.

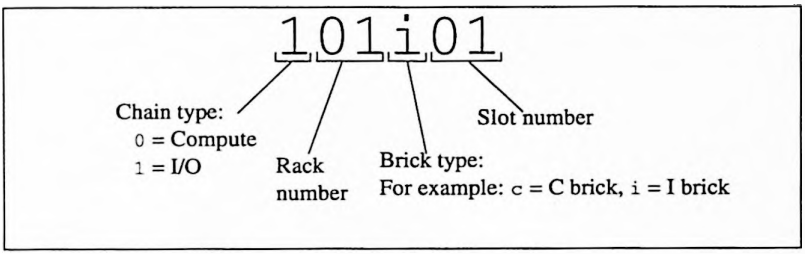

Figure 2-1 Error Output Format for Chains and Bricks

2.3.2 scantest Troubleshooting Tips

Use loopback cables and nonstandard configurations to perform additional troubleshooting, if necessary.

Table 2-3 scantest Troubleshooting Tips

Problem	Solution
Failure during the SIC selection process	Check the scan signals (TCK, TMS, TDI, TDO, and TRST) that go to the SIC and the SIC address lines
Failure during the interconnect test	Identify and correct the problem before you run `slit`
Failure during the register level tests	Check the scan signals (TCK, TMS, TDI, TDO, and TRST) at each component in the chain

2.4 slit (Scan-based Link-level Protocol Interconnect Test)

The `slit` test uses extra boundary scan registers in the router, hub, and Xbridge ASICs to exercise router links at speed.

```
$SCAN_PATH_BIN/slit [-h] <-f <$SCAN_PATH_CONFIG/file>>
<-i <$SCAN_PATH_CONFIG/file>> [-d <file>] [-p <num>]
[-e <num>] [-t <test>] [-m <mode>] [-l <level>] [-C]
[-s <mode>] [-L <pats>] [-F <bit_mask>] [-R] [-v]
```

Table 2-4 slit Command-line Options

Option	Description
`-h`	Displays all available command-line options and exits
`-f` *<file>*	Uses the specified configuration file (Refer to Section 2.2, "Scan Tool Configuration Files," for valid `slit` configuration files; default: `slit.cfg`)
`-i` *<file>*	Inputs data from the specified file (Refer to Section 2.4.1, " slit Input Files," for valid `slit` input files; default `slit.in`)
`-d` *<file>*	Sends output to the specified file (default: `slit.dmp`)
`-p` *<num>*	Performs the specified number of passes (`1-10000`; default: `1`)
`-e` *<num>*	Displays only the specified number of errors (`0-10000`; default: `200`)
`-t` *<test>*	Runs the specified tests `irp` = instruction register path test `irl` = instruction register length test `ir` = instruction register path and length tests `prp` = bypass register path test `prl` = bypass register length test `pr` = bypass register path and length tests `lrp` = LLP register path test `lrl` = LLP register length test `lr` = LLP register path and length tests `llp` = LLP at-speed interconnect test `id` = read and display device ID registers `all` = all tests (Use + to select multiple tests; default: `irl+lrl+llp`)
`-m` *<mode>*	Uses the specified mode for the register tests `normal` = normal lengths and minimal pattern(s) (default) `elen` = extended lengths `epat` = extended pattern set

Table 2-4 (continued) slit Command-line Options

Option	Description
`-l` *<level>*	Selects the level of the register tests `chn` = chain level (default) `brd` = board level `chip` = chip level
`-C`	Selects an individual SIC to run the test
`-s` *<mode>*	Selects a stepping mode `no` = does not perform test stepping; runs continuously (default) `all` = steps through each pattern; prompts user `err` = steps through the failing pattern; prompts user
`-L` *<pats>*	Uses the specified LLP test pattern `detect` = patterns to detect interconnect faults (default) `isolate` = patterns to isolate short/bridging faults `stress` = patterns to stress the interconnects `custom` = user-defined patterns
`-F` *<bit_mask>*	Captures and compares the specified LLP flits (`0x01-0xFF`; default: `0xFF`)
`-R`	Disables use of resets
`-v`	Selects the verbose mode

2.4.1 slit Input Files

The input files for `slit` are available in the `/stand/sysco/cfg/scan/` directory.

Table 2-5 slit Input Files

File	Description
`c001.links`	P1 and pilot C-brick link definitions
`i001.links`	P1 I-brick link definitions
`i002.links`	Pilot I-brick link definitions
`p001.links`	P1 and pilot P-brick link definitions
`r001.links`	P1 and pilot R-brick link definitions
`x001.links`	P1 and pilot X-brick link definitions

2.4.2 slit Error Output Format

```
   Failure information for scan-based LLP interconnect test:
❶ Pattern(8-bits each) : ---01--- ---02--- ---03--- ---04---
❷ Actual data          : 01101010 0000.... 0000.... 10000000
❸ Difference data      :    ^^      ^^ ^     ^^ ^       ^ ^  ^
❹ Logical description  : 101p01:U0:PORT_A_OUT_DATA_6
[101p01:U0:PORT_A_IN_DATA_6]
```

❶ The pattern that `slit` was using when it detected the miscompare.

❷ The actual data that the system read.

❸ The difference data marks each data pattern in the signature with a ^ symbol where the actual data did not match the expected data.

❹ The logical description of the ports on the failing connection (*<chain and brick>*`:`*<chip>*`:`*<pin>*); the node shown in `[]` detected the miscompare. Refer to Figure 2-1 for chain and brick decoding information and Figure 2-2 for pin decoding information.

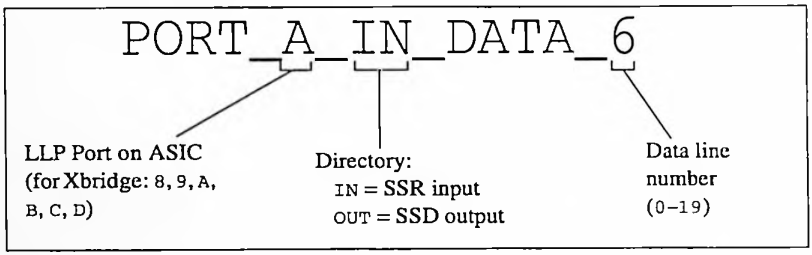

Figure 2-2 Error Output Format for Pins

2.4.3 slit Troubleshooting Tips

Use loopback cables and nonstandard configurations to perform additional troubleshooting, if necessary.

Table 2-6 slit Troubleshooting Tips

Problem	Solution
Fault detected between the A and B ports	Isolate the errors to a specific port by using a single-wrap cable or loopback cable on each port; replace the HIC on a failing port to isolate the error to the motherboard or HIC.
Multiple failures are detected on the A or B ports	Double-check the seating of the HIC and cable.
Entire port fails (20 data lines, 2 clock lines, and data ready) and all of the inputs are sampling zero (0)	Check the Widget Present signal.
`slit` LLP register test fails, but `scantest` passes	Check the system clock.
LLP-register interconnect test does not capture valid data for any pattern	Verify that both ports are configured with the same speed by using the L1 system controller; then, verify that the clocks are running.

2.5 scantool (Scan Software Scripting Application)

The `scantool` application is a scripting language that is used to run the scan tests.

`$SCAN_PATH_BIN/scantool [-l] [-c $SCAN_PATH_CFG/<config_file>] [-f $SCAN_PATH_DATA/<script_name>]`

Table 2-7 scantool Command-line Options

Option	Description
`-l`	Selects the L1 controller path
`-c <config_file>`	Uses the specified configuration file (Refer to Table 2-8 for valid `scantool` configuration files.)
`-f <script_name>`	Runs the specified script (`c_debug.tcl`, `c_internal.tcl`, `c_llp.tcl`, `c_membist.tcl`, `isttools.tcl`, `r_debug.tcl`, `r_internal.tcl`, `r_stopclk.tclp`)

Table 2-8 scantool Configuration Files

File	Description
`001c10_st.cfg`	Targets the C brick in slot 10 of rack 1
`001c13_st.cfg`	Targets the C brick in slot 13 of rack 1
`001c16_st.cfg`	Targets the C brick in slot 16 of rack 1
`001c21_st.cfg`	Targets the C brick in slot 21 of rack 1
`001c24_st.cfg`	Targets the C brick in slot 24 of rack 1
`001c29_st.cfg`	Targets the C brick in slot 29 of rack 1
`001c32_st.cfg`	Targets the C brick in slot 32 of rack 1
`001c35_st.cfg`	Targets the C brick in slot 35 of rack 1
`001r19_st.cfg`	Targets the R brick in slot 19 of rack 1
`001r27_st.cfg`	Targets the R brick in slot 27 of rack 1

Table 2-9 scantool Script Commands

Command	Description
SEL	Selects an object
DSEL	Deselects an object
INST	Sets the current instruction
GOINST	Scans the current instruction into a scan chain
DATA	Sets the current data pattern or read data
GODATA	Scans the current data pattern into a scan chain
REG	Specifies a user-defined data pattern
REGDATA	Sets the user-defined data
RESET	Resets the TAP controller(s) on the current scan chain
SIC	Causes a *family* to access and manipulate the SIC
GORUN	Moves the TAP controller to the Run-Test-Idle state and toggles the TCK signal
GOIDLE	Moves the TAP controller to the Run-Test-Idle state
INFO	Causes a *family* to extract information from an entity
SIMACC	Causes a *family* to control a Verilog simulation
EXIT	Exits the script

2.6 Scan Debugging Software

Scan debugging software includes commands that enable advanced users to perform *low-level* scan operations, which can be used for troubleshooting.

Caution: Use these commands only if you have detailed knowledge of the commands and the scan hardware.

Table 2-10 Scan Debugging Commands

Command	Description
`bit`	Displays the value of a bit located at a specified offset in a scan chain
`cleancc`	Removes the current context files for all inactive processes
`creg`	Compares the data for 2 logical registers and displays the differences (and the physical information for the miscomparing bits)
`errortoreport`	Converts an SVF error file format to an SGI report file format
`iort`	Uses the `sicio` command to test the input/output register in a SIC
`lts`	Extracts scan information from a log file and formats it for insertion into the gate-level simulator
`mervec`	Merges multiple sets of SGI format interconnect vector files into one file
`mkreg`	Creates a logical register definition file from HSDL/BSDL information in the scan database
`mkscan`	Builds scan commands from the latest versions in the source tree
`mksdb`	Builds a scan database from various source files
`module`	Selects the module port to use on the scan multiplexer box
`nic`	Extracts the number in a can (NIC) information from a device
`preg`	Displays the specified register object from a scan database
`qsdb`	Queries the scan database and displays the requested information
`scanrst`	Resets the selected scan subsystem
`sdbtodcd`	Builds a DCD format file from the HSDL information in a scan database
`sdc`	Provides direct control of each transition of the JTAG interface signals
`semrm`	Removes a scan semaphore that remains from a scan process
`setcc`	Sets the scan current context for other scan debugging commands to use as their default

Table 2-10 (continued) Scan Debugging Commands

Command	Description
`sicio`	Controls reads and writes of the SIC input/output register
`sicmr`	Controls reads and writes of the SIC mode register
`slpa`	Displays the internal scan state in the log file
`solc`	Displays the scan chain data from the target scan chain in the log file
`soll`	Displays the scan chain data bit from the target scan chain in the log file
`solr`	Displays the specified data from the target scan chain(s) specified by the logical register in the log file
`strip_for_bit`	Strips extraneous information from the `solc` command display to prepare it for use by the `bit` command
`tap`	Provides direct control over each transition of the JTAG interface signals (Provides higher level operations than the `sdc` command)
`topology`	Reads the scan topology from the selected scan database and displays it or writes it to a file
`tsic`	Tests the functionality of the SIC by shifting patterns through various internal registers
`tsmdb`	Selects all ports in a scan multiplexer box at one time to test daughter card functionality
`tsn`	Toggles the selected JTAG signal by using the `sdc` command

Chapter 3

Power-on Diagnostics and Discovery

Power-on diagnostics (POD) test the hardware needed to boot the system. Power-on diagnostics run automatically, but you can run several of the diagnostics manually also.

Discovery is the process that determines the configuration and interconnect topology of all the bricks in the system.

3.1 POD Mode Return Codes

Table 3-1 diag_rc Codes

Code	Description
0	No error occurred; the test completed successfully.
L1 and L2 Cache Error Codes	
1	Failed D cache 1 data test way 0
2	Failed D cache 1 data test way 1
3	Failed D cache 1 address test way 0
4	Failed D cache 1 address test way 1
5	Failed S cache 1 data test way 0
6	Failed S cache 1 data test way 1
7	Failed S cache 1 address test way 0
8	Failed S cache 1 address test way 1
9	Failed I cache data test way 0
10	Failed I cache data test way 1
11	Failed I cache address test way 0
12	Failed I cache address test way 1
13	Primary data cache test hung
14	Secondary cache test hung
15	Primary instruction cache test hung
16	Cache initialization
17	D cache tag RAM test
18	S cache tag RAM test way 0
19	S cache tag RAM test way 1
20	FPU S cache tag RAM test

Table 3-1 (continued) diag_rc Codes

Code	Description
21	Starting primary data cache test
22	Starting primary instruction cache test
23	Starting secondary cache test
24	Invalidate I and D caches
25	Invalidate I and D caches
26	Generic primary data cache test failure
27	Generic primary instruction cache test failure
CPU Error Codes	
30	The other CPU on this node failed
31	The CPU is disabled
32	The CPU failed in earlyinit
33	One of the CPUs took an unexpected exception
34	Coprocessor 1 is dead
35	The CPU died before or during local arbitration
36	The CPU and PROM mode bits do not match
Memory Error Codes	
40	The checksum in the PROM is bad
41	The copy of the PROM in memory is bad
42	The copy of PROM to memory failed
43	The `memtest` failed for one or more banks
44	The `dirtest` failed for one or more banks
45	Memory is disabled by the user
46	Bank 0 is unusable
47	Unsupported 256-MB bank on IP27
48	Failed memory bist test for some bank
IO7 Error Codes	
50	The `dgxbow` test failed
51	Xbridge sanity on IO7 failed
52	The IOC3 base address could not be found
53	Configuration space on IO7 failed
54	The `dgpci` test failed
55	Characters already exist in port A

Table 3-1 (continued) diag_rc Codes

Code	Description
56	Characters already exist in port B
57	Unable to transmit characters on port A
58	Unable to transmit characters on port B
59	Miscompares occurred on port A
60	Miscompares occurred on port B
61	Miscompares occurred on port A in DMA mode
62	Miscompares occurred on port B in DMA mode
63	A SCSI mailbox failure occurred
64	Unable to load words into the SSRAM
65	Unable to read words out of the SSRAM
66	Miscompares occurred on the SCSI SSRAM
67	Unable to load words into the SSRAM
68	Unable to dump words from the SSRAM
69	DMA miscompares from the SSRAM occurred
70	Unable to load the self-test firmware
71	Unable to execute the `controller` test
72	The `controller` test failed with miscompares
80	An exception occurred on the `dgxbow` test
81	An Xbridge exception occurred
82	An IO7 configuration exception occurred
83	A PCI bus exception occurred
84	A serial PIO exception occurred
85	A serial DMA exception occurred
86	A SCSI RAM exception occurred
87	A SCSI DMA exception occurred
88	A SCSI controller exception occurred
101	`ENET_EXCEPTION`
102	`ENET_SSRAM_FAIL`
103	`ENET_PHYREG_FAIL`
104	`ENET_PHY_R_BUSY_TO`
105	`ENET_PHY_W_BUSY_TO`
106	`ENET_PHY_RESET_TO`

Table 3-1 (continued) diag_rc Codes

Code	Description
108	ENET_LOOP_ILL_PARM
109	ENET_LOOP_MALLOC
110	ENET_EMCR_RST_TO
111	ENET_EMCR_RXEN_TO
112	ENET_LOOP_AN_TO
113	ENET_LOOP_AN_ABLE
114	ENET_LOOP_AN_MODE
115	ENET_LOOP_DMA_TO1
116	ENET_LOOP_DMA_TO2
117	ENET_LOOP_DMA_TO3
118	ENET_LOOP_BAD_RUPT1
119	ENET_LOOP_BAD_RUPT2
120	ENET_LOOP_BAD_RUPT3
121	ENET_LOOP_BAD_RUPT4
122	ENET_LOOP_NO_RUPT
123	ENET_LOOP_BAD_W1
124	ENET_LOOP_BAD_STAT
125	ENET_LOOP_BAD_DATA1
126	ENET_LOOP_BAD_DATA2
150	Too many link errors occurred in the hub NI
151	Too many miscellaneous errors from the router ASIC occurred in the hub NI
152	The hub NI did not detect forced errors
153	The hub chip failed the lbist test
154	The hub chip failed the abist test
155	The LLP interface on the hub chip is down
156	The hub ASIC could not launch discovery
157	The interrupt test failed for the hub chip
158	An NI error occurred in the kernel
159	The BTE test failed for the hub chip
160	The BTE for the hub chip hung while diagnostics were running
161	The hub ASIC has a real-time clock error

Table 3-1 (continued) diag_rc Codes

Code	Description
162	A reset propagation error occurred
Router Error Codes	
200	The hub ASIC did not get a response from a router ASIC
201	The hub ASIC received an overrun/mismatch reply from a router ASIC
202	The hub ASIC encountered a hardware error while accessing a router ASIC
203	The hub ASIC received a protection error from a router ASIC
204	The hub ASIC received an invalid vector error from a router ASIC
205	The router ASIC did not detect forced errors
206	The router ASIC detected too many link errors
207	The router ASIC failed the lbist test
208	The router ASIC failed the abist test
209	FRU analysis for the local node is available
210	FRU analysis for all nodes is available
211	RTC distribution failed
Generic Error Codes	
240	No CPU is available
241	Returning to POD mode
242–247	A PROM panic occurred
248–252	An NMI occurred
253	Switching to POD mode at the request of the user

3.2 IP35 PROM Tests

The processor and memory power-on diagnostics can be run manually from the POD mode command line (POD>). To enter POD mode, enter the `pod` command from the PROM monitor prompt. Then, enter the command of the test that you want to run.

3.2.1 IP35 PROM Test Failure LEDs

Table 3-2 IP35 PROM Test Failure LEDs

Hex Value	LED Label	Description
0x81	FLED_CP1	Register test failed
0x83	FLED_ICACHE	Primary instruction cache test failed
0x84	FLED_DCACHE	Primary data cache test failed
0x85	FLED_SCACHE	Secondary cache test failed
0x86	FLED_KILLED	CPU disabled by another
0x87	FLED_RTC	Real-time counter failure
0x88	FLED_ECC	Error-correction code failure
0x89	FLED_XTLBMISS	XTLB miss exception
0x8A	FLED_UTLBMISS	UTLB miss exception
0x8B	FLED_KTLBMISS	KTLB miss exception
0x8C	FLED_GENERAL	General exception
0x90	FLED_HUBINTS	Interrupt test failed
0x91	FLED_HUBLOCAL	Hub local test failed
0x92	FLED_HUBCONFIG	Hub configuration test failed
0x95	FLED_HUBUART	Hub UART test failed
0x98	FLED_NOMEM	No local memory
0xA9	FLED_BADMEM	No good local memory

3.2.2 IP35 PROM Test Error Messages

Table 3-3 IP35 PROM Test Error Messages

Message	Description
`RSLT test_hub_error_detection FAIL`	Hub error detection test failed
`RSLT rtr_send_data_error FAIL`	Router send-data error test failed
`RSLT test_rtr_error_detection FAIL`	Router error detection test failed
`RSLT get_chipid FAIL`	Get chipid test failed

Table 3-3 (continued) IP35 PROM Test Error Messages

Message	Description
`RSLT xbow_sanity FAIL`	Xbridge XIO test failed
`RSLT bridge_sanity FAIL`	Xbridge test failed
`RSLT io7config_space FAIL`	IO7 configuration space test failed
`RSLT pcibus_sanity FAIL`	PCI bus test failed
`RSLT serial_pio FAIL`	Serial programmed I/O test failed
`RSLT serial_dma FAIL`	Serial DMA test failed
`RSLT rt_clock_test FAIL`	GClk logic test failed
`RSLT hub_bte_diag FAIL`	BTE test failed
`RSLT prom_checksum FAIL`	PROM checksum test failed
`RSLT dirtest FAIL` or `MEMORY FAILURE`	Backdoor directory space test failed
`RSLT memtest FAIL` or `MEMORY FAILURE`	Memory test failed
`RSLT chk_local_link FAIL`	Local link connection test failed
`RSLT hub_send_data_error FAIL`	Hub send-data error test failed
`No Memory Found` or `*** Mixed standard and premium memory` `*** Treating all as standard.` or `nasid` *`node_number`*`: disabling bank` *`bank_number`*	Memory configuration test failed
`RSLT cache_test_s FAIL`	Secondary cache test failed
`RSLT hub_int_diag FAIL`	Interrupt test failed

3.2.3 Block Transfer Engine Test

The block transfer engine (BTE) test checks the BTE logic in the hub ASIC. This test cannot be run manually.

Table 3-4 BTE Test Errors

diag_rc	Message	Description
159	KLDIAG_HUB_BTE_FAILED	BTE test failed on the hub chip
160	KLDIAG_HUB_BTE_HUNG	The BTE of the hub chip hung during diagnostics

Failure Message: `RSLT hub_bte_diag FAIL`

FRU: IP35 motherboard

3.2.4 chklink (Local Link Connection Test)

The `chklink` test checks the connection between the local hub ASIC and an attached hub or router ASIC. This test can be run manually by using the following command:

```
POD> chklink
```

Failure Message: `RSLT chk_local_link FAIL`

FRU: IP35 motherboard

3.2.5 dgbrdg (Xbridge Test)

The `dgbrdg` test checks for valid reset values in the Xbridge registers and tests the Xbridge interrupt hardware. This test can be run manually by using the following command:

`POD> dgbrdg [mn|mh|mm] [n<nasid>] [b<bus>]`

Table 3-5 dgbrdg Command-line Options

Option	Description
mn | mh | mm	Runs the test in the specified mode mn = normal (default) mh = heavy mm = manufacturing
n<*nasid*>	Runs the test on the node with the specified NASID value (default: node from which you entered the command)
b<*bus*>	Tests the specified PCI bus

Table 3-6 dgbrdg Errors

diag_rc	Message	Description
51	KLDIAG_BRIDGE_FAIL	Xbridge sanity on IO7 failed
81	KLDIAG_BRIDGE_EXCEPTION	

Failure Message: `RSLT bridge_sanity FAIL`

FRU: IP35 motherboard

3.2.6 dgconf (IO7 Configuration Space Test)

The `dgconf` test verifies the PCI configuration space on the BaseIO board and checks for the presence of the USB and FireWire connections. This test can be run manually by using the following command:

```
POD> dgconf [mn|mh|mm] [n<nasid>] [b<bus>]
```

Table 3-7 dgconf Command-line Options

Option	Description
mn | mh | mm	Runs the test in the specified mode mn = normal (default) mh = heavy mm = manufacturing
n<*nasid*>	Runs the test on the node with the specified NASID value (default: node from which you entered the command)
b<*bus*>	Tests the specified PCI bus

Table 3-8 dgconf Errors

diag_rc	Message	Description
53	KLDIAG_PCICONFIG_FAIL	IO7 configuration space test failed
82	KLDIAG_IO7CONFIG_EXCEPTION	

Failure Message: `RSLT io7config_space FAIL`

FRU: BaseIO board

3.2.7 dgpci (PCI Bus Test)

The `dgpci` test checks the PCI bus with walking data tests. This test can be run manually by using the following command:

`POD> dgpci [mn|mh|mm] [n<nasid>] [b<bus>] [p<PCINUM>]`

Table 3-9 dgpci Command-line Options

Option	Description
mn | mh | mm	Runs the test in the specified mode mn = normal (default) mh = heavy mm = manufacturing
n<*nasid*>	Runs the test on the node with the specified NASID value (default: node from which you entered the command)
b<*bus*>	Tests the specified PCI bus
p<*PCINUM*>	Tests the specified IOC3 ASIC (default: 2 = IOC3 chip on the BaseIO board

Table 3-10 dgpci Errors

diag_rc	Message	Description
54	KLDIAG_PCIBUS_FAIL	PCI bus test failed
83	KLDIAG_PCIBUS_EXCEPTION	

Failure Message: `RSLT pcibus_sanity FAIL`

FRU: BaseIO board

3.2.8 dgsdma (Serial DMA Test)

The `dgsdma` test verifies that serial DMA can be performed. This test can be run manually by using the following command:

`POD> dgsdma [mn|mh|mm|mx] [n<nasid>] [b<bus>] [p<PCINUM>]`

Table 3-11 dgsdma Command-line Options

Option	Description
mn | mh | mm | mx	Runs the test in the specified mode mn = normal (default) mh = heavy mm = manufacturing mx = external loopback
n<*nasid*>	Runs the test on the node with the specified NASID value (default: node from which you entered the command)
b<*bus*>	Tests the specified PCI bus
p<*PCINUM*>	Tests the specified IOC3 ASIC (default: 2 = IOC3 ASIC on the BaseIO board)

Table 3-12 dgsdma Errors

diag_rc	Message	Description
61	KLDIAG_SER_DMA_FAILA	Miscompares on A in DMA mode
62	KLDIAG_SER_DMA_FAILB	Miscompares on B in DMA mode
85	KLDIAG_SERDMA_EXCEPTION	

Failure Message: `RSLT serial_dma FAIL`

FRU: BaseIO board

3.2.9 dgspio (Serial Programmed I/O Test)

The `dgspio` test verifies the serial PIO functionality of the BaseIO board. This test can be run manually by using the following command:

`POD> dgspio [mn|mh|mm|mx] [n<nasid>] [b<bus>] [p<PCINUM>]`

Table 3-13 dgspio Command-line Options

Option	Description
mn | mh | mm | mx	Runs the test in the specified mode mn = normal (default) mh = heavy mm = manufacturing mx = external loopback
n<*nasid*>	Runs the test on the node with the specified NASID value (default: node from which you entered the command)
b<*bus*>	Tests the specified PCI bus
p<*PCINUM*>	Tests the specified IOC3 ASIC (default: 2 = IOC3 ASIC on the BaseIO board)

Table 3-14 dgspio Errors

diag_rc	Message	Description
55	KLDIAG_SER_PIO_FAIL1A	Characters already exist in port A
56	KLDIAG_SER_PIO_FAIL1B	Characters already exist in port B
57	KLDIAG_SER_PIO_FAIL2A	Unable to transmit characters on port A
58	KLDIAG_SER_PIO_FAIL2B	Unable to transmit characters on port B
59	KLDIAG_SER_PIO_FAILA	Miscompares on port A
60	KLDIAG_SER_PIO_FAILB	Miscompares on port B
84	KLDIAG_SERPIO_EXCEPTION	

Failure Message: `RSLT serial_pio FAIL`

FRU: BaseIO board

3.2.10 dgxbow (Xbridge XIO Test)

The dgxbow test checks for valid reset values in the Xbridge XIO registers and tests the XIO interrupt hardware. This test can be run manually by using the following command:

```
POD> dgxbow [mn|mh|mm] [n<nasid>]
```

Table 3-15 dgxbow Command-line Options

Option	Description
mn | mh | mm	Runs the test in the specified mode mn = normal (default) mh = heavy mm = manufacturing
n<*nasid*>	Specifies the node on which the test should run (default: node from which you entered the command)

Table 3-16 dgxbow Errors

diag_rc	Message	Description
50	KLDIAG_XBOW_FAIL	The dgxbow test failed
80	KLDIAG_XBOW_EXCEPTION	An exception on the dgxbow test occurred

Failure Message: `RSLT xbow_sanity FAIL`

FRU: C brick

3.2.11 dirtest (Backdoor Directory Space Test)

The `dirtest` test checks the memory that contains the backdoor directory space. This test can be run manually by using the following command:

`POD> dirtest` *<START> <LEN>*

Table 3-17 dirtest Command-line Options

Option	Description
START	Specifies a memory address that corresponds to a directory address (multiples of 4,096—0x1000)
LEN	Specifies the size of the area to test (multiples of 4,096—0x1000)

Failure diag_rc: 44 `KLDIAG_DIRTEST_FAILED`

Failure Message: `RSLT dirtest FAIL` or `MEMORY FAILURE`

FRU: DIMM

3.2.12 Get Chipid Test

The get chipid test obtains the ID of a hub or router chip that is attached across an LLP link. This test cannot be run manually.

Failure Message: `RSLT get_chipid FAIL`

FRU: router ASIC or IP35 motherboard

3.2.13 Global Clock Logic Test

The Global Clock (GClk) logic test checks the functionality of the hub ASIC global clock logic. This test cannot be run manually.

Failure Message: `RSLT rt_clock_test FAIL`

FRU: IP35 motherboard

3.2.14 Hub Configuration Test

The hub configuration test tests the hub ASIC. This test cannot be run manually.

Failure LED: 0x92 `FLED_HUBCONFIG`

FRU: IP35 motherboard

3.2.15 Hub Local Test

The hub local test tests the hub ASIC. This test cannot be run manually.

Failure LED: 0x91 `FLED_HUBLOCAL`

FRU: IP35 motherboard

3.2.16 Hub UART Test

The hub UART test tests the hub ASIC. This test cannot be run manually.

Failure LED: 0x95 `FLED_HUBUART`

FRU: IP35 motherboard

3.2.17 hubsde (Hub Send-data Error Test)

The `hubsde` test verifies that the hub or router ASIC that is attached to the local hub ASIC can detect link errors. This test can be run manually by using the following command:

```
POD> hubsde
```

Failure Message: `RSLT hub_send_data_error FAIL`

FRU: router ASIC, IP35 motherboard, or cable

3.2.18 Hub Error Detection Test

The hub error detection test is a recursive version of the `hubsde` test. This test cannot be run manually.

Failure Message: `RSLT test_hub_error_detection FAIL`

FRU: router ASIC, IP35 motherboard, or cable

3.2.19 Interrupt Test

The interrupt test checks the hub chip interrupt registers and verifies that the processors can correctly receive interrupts. This test cannot be run manually.

Failure diag_rc: 157 `KLDIAG_HUB_INTS_FAILED`

Failure LED: 0x90 `FLED_HUBINTS`

Failure Message: `RSLT hub_int_diag FAIL`

FRU: IP35 motherboard

3.2.20 Memory Configuration Test

The memory configuration test verifies that the memory configuration of the system is valid. This test cannot be run manually.

Table 3-18 Memory Configuration Test Errors

LED	Message	Description
0x98	`FLED_NOMEM`	no local memory
0xA9	`FLED_BADMEM`	no good local memory

Failure Message: `No Memory Found` or
`*** Mixed standard and premium memory`
`*** Treating all as standard.` or
`nasid` *`node_number`*`: disabling bank` *`bank_number`*

FRU: DIMM

3.2.21 memtest (Memory Test)

The `memtest` test checks the memory address and data lines. This test can be run manually by using the following command:

`POD> memtest <START> <LEN>`

Table 3-19 memtest Command-line Options

Option	Description
START	Specifies the starting address of the memory range (multiples of 4,096—0x1000)
LEN	Specifies the size of the range to test (multiples of 4,096—0x1000)

Failure diag_rc: 43 `KLDIAG_MEMTEST_FAILED`

Failure Message: `RSLT memtest FAIL` or `MEMORY FAILURE`

FRU: DIMM

3.2.22 Primary Data Cache Test

The primary data cache test writes to and reads from the primary data cache. This test cannot be run manually.

Table 3-20 Primary Data Cache Test Errors

diag_rc	Message	Description
1	`KLDIAG_DCACHE_DATA_WAY0`	Failed D cache 1 data test way 0
2	`KLDIAG_DCACHE_DATA_WAY1`	Failed D cache 1 data test way 1
3	`KLDIAG_DCACHE_ADDR_WAY0`	Failed D cache 1 address test way 0
4	`KLDIAG_DCACHE_ADDR_WAY1`	Failed D cache 1 address test way 1
13	`KLDIAG_DCACHE_HANG`	D cache test hung
26	`KLDIAG_DCACHE_FAIL`	Generic D cache test failure

Failure LED: 0x84 `FLED_DCACHE`

FRU: PIMM

3.2.23 Primary Instruction Cache Test

The primary instruction cache test writes to and reads from the primary instruction cache. This test cannot be run manually.

Table 3-21 Primary Instruction Cache Test Errors

diag_rc	Message	Description
9	KLDIAG_ICACHE_DATA_WAY0	Failed I cache data test way 0
10	KLDIAG_ICACHE_DATA_WAY1	Failed I cache data test way 1
11	KLDIAG_ICACHE_ADDR_WAY0	Failed I cache address test way 0
12	KLDIAG_ICACHE_ADDR_WAY1	Failed I cache address test way 1
15	KLDIAG_ICACHE_HANG	I cache test hung
27	KLDIAG_ICACHE_FAIL	Generic I cache test failure

Failure LED: 0x83 FLED_ICACHE

FRU: PIMM

3.2.24 PROM Checksum Test

The PROM checksum test verifies the copy of PROM code in memory. This test cannot be run manually.

Failure diag_rc: 40 KLDIAG_PROM_CHKSUM_BAD

Failure Message: RSLT prom_checksum FAIL

FRU: IP35 motherboard

3.2.25 Register Test

The register test checks for stuck bits in the processor registers. This test cannot be run manually.

Failure LED: 0x81 FLED_CP1

FRU: PIMM

3.2.26 rtrsde (Router Send-data Error Test)

The `rtrsde` test verifies that the local hub ASIC can detect link errors. This test can be run manually by using the following command:

```
POD> rtrsde
```

Failure Message: `RSLT rtr_send_data_error FAIL`

FRU: router ASIC, IP35 motherboard, or cable

3.2.27 Router Error Detection Test

The router error detection test is a recursive version of the `rtrsde` test. This test cannot be run manually.

Failure diag_rc: 205 `KLDIAG_RTR_FAIL_ERR_DET`

Failure Message: `RSLT test_rtr_error_detection FAIL`

FRU: router ASIC, IP35 motherboard, or cable

3.2.28 Secondary Cache Test

The secondary cache test checks for stuck bits in the secondary cache SRAMs. This test cannot be run manually.

Table 3-22 Secondary Cache Test Errors

Value	Message	Description
diag_rc Values		
5	KLDIAG_SCACHE_DATA_WAY0	Failed S cache 1 data test way 0
6	KLDIAG_SCACHE_DATA_WAY1	Failed S cache 1 data test way 1
7	KLDIAG_SCACHE_ADDR_WAY0	Failed S cache 1 address test way 0
8	KLDIAG_SCACHE_ADDR_WAY1	Failed S cache 1 address test way 1
14	KLDIAG_SCACHE_HANG	S cache test hung
LED Values		
0x85	FLED_SCACHE	Secondary cache test failed
0x86	FLED_KILLED	CPU disabled by another
0x87	FLED_RTC	Real-time counter failure
0x88	FLED_ECC	Error-correction code failure
0x89	FLED_XTLBMISS	XTLB miss exception
0x8A	FLED_UTLBMISS	UTLB miss exception
0x8B	FLED_KTLBMISS	KTLB miss exception
0x8C	FLED_GENERAL	General exception

Failure Message: `RSLT cache_test_s FAIL`

FRU: PIMM

3.3 BaseIO PROM Tests

From the command monitor prompt (>>), you can manually run the I/O path power-on diagnostics.

3.3.1 BaseIO PROM Test Error Messages

Table 3-23 BaseIO PROM Test Error Messages

Message	Description
`RSLT enet_all FAIL`	Comprehensive Ethernet test failed
`RSLT enet_ssram FAIL`	Ethernet SSRAM test failed
`RSLT enet_phy_reg FAIL`	PHY register test failed
`RSLT enet_ioc3_loop FAIL`	IOC3 internal loopback DMA test failed
`RSLT enet_phy_reg FAIL`	PHY chip internal loopback test failed
`RSLT enet_tw_loop FAIL`	Twister chip internal loopback test failed
`RSLT enet_ext_loop FAIL`	External loopback DMA test failed
`RSLT enet_10_ext_loop FAIL`	External loopback DMA 10-Mbps test failed
`RSLT enet_100_ext_loop FAIL`	External loopback DMA 100-Mbps test failed
`RSLT enet_xtalk_stress FAIL`	Xtalk stress test failed
`RSLT scsi_ram FAIL`	SCSI SSRAM test failed
`RSLT scsi_dma FAIL`	SCSI DMA test failed
`RSLT scsi_controller FAIL`	SCSI self-test failed

3.3.2 diag_enet (Comprehensive Ethernet Test)

The `diag_enet` test runs all of the Ethernet tests in an order that optimizes fault isolation. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e] [-v] [-t <testname>] [-r <count>]
[-R <count>]
```

Table 3-24 diag_enet Command-line Options

Option	Description
`-N` *<nasid>*	Tests the module with the specified NASID number (default: console BaseIO module)
`-m` *<moduleid>*	Tests the module with the specified module number (default: console BaseIO module)
`-b` *<bus>*	Tests the specified PCI bus
`-p` *<pcidev>*	Specifies the PCI device number of the target IOC3 (default: 2 = IOC3 on a BaseIO board)
`-h`	Runs the diagnostic in heavy mode
`-e`	Runs the diagnostic in external loopback mode
`-v`	Runs the diagnostic in verbose mode
`-t` *<testname>*	Runs only the specified test (Refer to Sections 3.3.2.1 through 3.3.2.9 for valid `diag_enet` tests; default: all)
`-r` *<count>*	Repeats the test for the specified number of times and stops when it detects a failure
`-R` *<count>*	Repeats the test for the specified number of times and continues when it detects a failure

Failure diag_rc: 101 `KLDIAG_ENET_EXCEPTION`

Failure Message: `RSLT enet_all FAIL`

FRU: BaseIO (I brick)

3.3.2.1 ext_loop (External Loopback DMA Test)

The `ext_loop` test verifies the Ethernet DMA, the autonegotiation feature, and the DMA at 10 and 100 Mbps. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t ext_loop> [-r <count>]
[-R <count>]
```

Failure Message: `RSLT enet_ext_loop FAIL`

3.3.2.2 ext_loop_10 (External Loopback DMA 10-Mbps Test)

The `ext_loop_10` test verifies the DMA at 10 Mbps and does not test autonegotiation. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t ext_loop_10> [-r <count>]
[-R <count>]
```

Failure Message: `RSLT enet_10_ext_loop FAIL`

3.3.2.3 ext_loop_100 (External Loopback DMA 100-Mbps Test)

The `ext_loop_100` test verifies the DMA at 100 Mbps and does not test autonegotiation. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t ext_loop_100>
[-r <count>] [-R <count>]
```

Failure Message: `RSLT enet_100_ext_loop FAIL`

3.3.2.4 ioc3_loop (IOC3 Internal Loopback Test)

The `ioc3_loop` test verifies Ethernet DMA by using the internal loopback features of the IOC3 ASIC. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t ioc3_loop> [-r <count>]
[-R <count>]
```

Failure diag_rc: 52 `KLDIAG_IOC3_BASE_FAIL`

Failure Message: `RSLT enet_ioc3_loop FAIL`

3.3.2.5 phy_loop (PHY Chip Internal Loopback Test)

The `phy_loop` test checks Ethernet DMA. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t phy_loop> [-r <count>]
[-R <count>]
```

Failure Message: `RSLT enet_phy_reg FAIL`

3.3.2.6 phyreg (PHY Register Test)

The `phyreg` test checks the reset values of the registers on the PHY chip and verifies the capability to read and write selected bits in selected registers on the PHY chip. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t phyreg> [-r <count>]
[-R <count>]
```

Table 3-25 phyreg Errors

diag_rc	Message
103	KLDIAG_ENET_PHYREG_FAIL
104	KLDIAG_ENET_PHY_R_BUSY_TO
105	KLDIAG_ENET_PHY_W_BUSY_TO
106	KLDIAG_ENET_PHY_RESET_TO

Failure Message: `RSLT enet_phy_reg FAIL`

3.3.2.7 ssram (Ethernet SSRAM Test)

The `ssram` test checks the 128-KB Ethernet packet-buffering SSRAM, the interface of the SSRAM to the IOC3 ASIC, and the parity generation and checking logic in the IOC3. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t ssram> [-r <count>]
[-R <count>]
```

Failure diag_rc: 102 `KLDIAG_ENET_SSRAM_FAIL`

Failure Message: `RSLT enet_ssram FAIL`

3.3.2.8 tw_loop (Twister Chip Internal Loopback Test)

The `tw_loop` test verifies Ethernet DMA. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t tw_loop> [-r <count>]
[-R <count>]
```

Failure Message: `RSLT enet_tw_loop FAIL`

3.3.2.9 xtalk (Xtalk Stress Test)

The `xtalk` test checks the IOC3 and Xbridge interrupt registers for errors, but it does not check the received data. This test can be run manually by using the following command:

```
>> diag_enet [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e|-x] [-v] <-t xtalk> [-r <count>]
[-R <count>]
```

3.3.3 diag_fibre (Fibre Channel Test)

The `diag_fibre` test runs all of the SCSI tests to verify the Fibre Channel interface to the system boot disk. This test can be run manually by using the following command:

```
>> diag_fibre [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-e] [-v] [-t <testname>] [-r <count>]
[-R <count>]
```

Table 3-26 diag_fibre Command-line Options

Option	Description
`-N` *<nasid>*	Tests the module with the specified NASID number (default: console BaseIO module)
`-m` *<moduleid>*	Tests the module with the specified module number (default: console BaseIO module)
`-b` *<bus>*	Tests the specified PCI bus
`-p` *<pcidev>*	Specifies the PCI device number of the target IOC3 (default: 0 = first SCSI controller on a BaseIO board)
`-h`	Runs the diagnostic in heavy mode
`-e`	Runs the diagnostic in external loopback mode
`-v`	Runs the diagnostic in verbose mode
`-t` *<testname>*	Runs only the specified test (Refer to Sections 3.3.3.1 through 3.3.3.3 for valid `diag_fibre` tests; default: all.)
`-r` *<count>*	Repeats the test for the specified number of times and stops when it detects a failure
`-R` *<count>*	Repeats the test for the specified number of times and continues when it detects a failure

FRU: BaseIO (I brick)

3.3.3.1 ram (SCSI SSRAM)

The `ram` test verifies that data can be loaded into SCSI SSRAM memory and read back successfully. It also verifies the *mailbox* mechanism. This test can be run manually by using the following command:

```
>> diag_fibre [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-v] <-t ram> [-r <count>] [-R <count>]
```

Table 3-27 ram Errors

diag_rc	Message	Description
63	KLDIAG_SCSI_MBOX_FAIL	SCSI mailbox failure
64	KLDIAG_SCSI_SSRAM_FAIL1	Unable to load words into the SSRAM
65	KLDIAG_SCSI_SSRAM_FAIL2	Unable to read words out of SSRAM
66	KLDIAG_SCSI_SSRAM_FAIL	Miscompares on the SCSI SSRAM
86	KLDIAG_SCSIRAM_EXCEPTION	

3.3.3.2 dma (SCSI DMA)

The `dma` test verifies that the SCSI DMA mechanism is functional. This test can be run manually by using the following command:

```
>> diag_fibre [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-v] <-t dma> [-r <count>] [-R <count>]
```

Table 3-28 dma Errors

diag_rc	Message	Description
67	KLDIAG_SCSI_DMA_FAIL1	Unable to load words into SSRAM
68	KLDIAG_SCSI_DMA_FAIL2	Unable to dump words from SSRAM
69	KLDIAG_SCSI_DMA_FAIL	DMA miscompares from SSRAM
87	KLDIAG_SCSIDMA_EXCEPTION	

3.3.3.3 controller (SCSI Self-test)

The `controller` test uses the SCSI self-test firmware to test the SCSI controller. This test can be run manually by using the following command:

```
>> diag_fibre [-N <nasid>] [-m <moduleid>] [-b <bus>]
[-p <pcidev>] [-h] [-v] <-t controller> [-r <count>]
[-R <count>]
```

Table 3-29 controller Errors

diag_rc	Message	Description
70	KLDIAG_SCSI_STEST_FAIL1	Unable to load self-test firmware
71	KLDIAG_SCSI_STEST_FAIL2	Unable to execute SCSI self-test
72	KLDIAG_SCSI_STEST_FAIL	Self-test failed with miscompares
88	KLDIAG_SCSICNTRL_EXCEPTION	

Chapter 4

scalable Micro-diagnostic Kernel (sMDK)

The sMDK provides test access to hardware components that cannot be accessed from a user program that is running under the operating system.

4.1 sMDK Commands

Table 4-1 Useful sMDK Commands

Option	Description
`cfg`	Displays the hardware configuration of the system
`cfg i` *<io_brick>*	Displays the configuration of an I/O brick
`cfg n` *<vnode>*	Displays the configuration of a node
`cfg r` *<vrouter>*	Displays the configuration of a router ASIC
`drop` *<diag_index>*	Kills all processes that are spawned by the specified diagnostic
`ds`	Displays status information for all diagnostics that are running
`dscr` *<diag_index>*	Lists the displays that are available for the diagnostic specified by the diagnostic index (*diag_index*)
`dscr` *<diag_index> <screen> <page> <optarg0> <optarg1>*	Displays the diagnostic information for the specified page
`el`	Lists all nodes that have logged hardware-detected errors and the number of each type of error
`el` *<vnode>*	Displays detailed information about the hardware-detected errors that were logged by the specified node
`elclr`	Clears the error logs on all nodes
`help`	Lists all available sMDK commands
`help` *<command>*	Displays help information for a command
`kl`	Displays a list of virtual CPUs that have entries in their kernel logs
`kl` *<vcpu>*	Displays the kernel log for the specified virtual CPU
`klclr`	Clears all kernel logs
`load` *<filename> <options>*	Loads a test into memory and selects it as the current test

Table 4-1 (continued) Useful sMDK Commands

Option	Description
`lpath <load directory path>`	Specifies the location where the sMDK files are located (default: `/stand/smdk`)
`ps`	Displays the status of all processes
`ps <vcpu>`	Displays the process status for the specified virtual CPU
`reset`	Resets the system
`run`	Runs the current process
`run <test> <options>`	Loads a diagnostic from the host, selects it as the current process, and then runs the diagnostic
`version`	Displays the version of sMDK that you are using and the data and time that it was built

4.2 Running a Test

1. Start sMDK:
   ```
   >> boot -f dksc(0,1,0)/stand/smdk/smdk
   ```
   Note: Use the `hinv` command to determine the location of the system disk.

2. Load and run a test:
   ```
   smdk> run <test> [test_options]
   ```

3. View the test status and determine the index value:
   ```
   smdk> ds
   ```

4. Determine which displays are available by using the diagnostic test index:
   ```
   smdk> dscr <diag_index>
   ```

5. View a display:
   ```
   smdk> dscr <diag_index> <screen> <page>
   ```

6. Stop the test:
   ```
   smdk> drop <diag_index>
   ```

7. Reset the system:
   ```
   smdk> reset
   ```

4.3 Running a Utility

1. Start sMDK:
   `>> boot -f dksc(0,1,0)/stand/smdk/smdk`
2. Load and run a utility:
   `smdk> run <utility> [utility_options]`

4.4 barr (Barrier Circuitry Test)

The `barr` test checks the router ASIC local block registers that are used for barrier configuration and the state machine fan-in/fan-out logic that is used for barrier operation. SGI recommends that you run this test for at least 5000 passes (`run barr -e 5000`).

Table 4-2 barr Test Sections

Section	Description
0	Performs a basic barrier operation test
1	Performs a single node test
2	Performs a single router test
3	Tests the entire system

`barr [-S <section select>] [ -x <barrier contexts>] [-e <ending pass>] [-h|-H]`

Table 4-3 barr Command-line Options

Option	Description
`-S <section select>`	Runs the specified test section (4-bit hexadecimal bitmask; refer to Table 4-2 for valid `barr` test sections)
`-x <barrier contexts>`	Tests the specified barrier contexts (32-bit hexadecimal bitmask)
`-e <ending pass>`	Specifies the ending pass count (default: `1000`)
`-h|-H`	Displays help information

4.4.1 barr Troubleshooting Tips

When this test detects an error, the failing CPU halts, which may cause other CPUs to fail with synchronization errors.

4.5 io7bt (IO7 Test)

The `io7bt` test verifies all of the IO7 hardware that is embedded on the motherboard of an I brick. SGI recommends that you run this test for at least 10 passes (`run io7bt -e 10`).

Table 4-4 io7bt Test Sections

Section	Description
0	Performs a device access test
1	Verifies the IOC3 hardware
2	Verifies the USB hardware
3	Verifies the IEEE 1394 hardware
4	Tests concurrent data transfers

`io7bt [-C <value>] [-d <bit_mask>] [-h|-H] [-n <node>] [-p <port>] [-l <value>] [-s <start pass>] [-e <end_pass>] [-S <bit_mask>] [-E <bit_mask>] [-I <value>] [-N <value>] [-F <value>] [-U <value>]`

Table 4-5 io7bt Command-line Options

Option	Description
`-C` *<value>*	Performs the specified action when an error occurs 0 = continues testing 1 = stops testing 2 = fills the buffer and stops testing
`-d` *<bit_mask>*	Uses the specified device (bitmask) bit 1 = tests the IEEE 1394 hardware (slot 5) bit 2 = tests the USB hardware (slot 4) bit 3 = tests the IOC3 hardware (slot 0)
`-h|-H`	Displays help information
`-n` *<node>*	Tests the specified node
`-p` *<port>*	Tests the specified port
`-l` *<value>*	Selects the looping option 0 = loop mode off 1 = loop mode on (repeats the test based on the `-s` and `-e` values) **Note:** If the `-e` option is set to 0, this option is disabled.
`-s` *<start pass>*	Specifies the starting pass count (default: 0)
`-e` *<end_pass>*	Specifies the ending pass count (default: 0 = runs forever)
`-S` *<bit_mask>*	Runs the specified test sections (Refer to Table 4-4 for valid `io7bt` test sections.)

Table 4-5 (continued) io7bt Command-line Options

Option	Description
-E <*bit_mask*>	Indicates that a connector is attached to the Ethernet port bit 0 = connector is attached to Ethernet port bit 1 = external loopback connector is attached to Ethernet port bit 2 = connector is attached to Ethernet port bit 3 = connector is attached to Ethernet port
-I <*value*>	Indicates that a connector is attached to the SuperIO port bit 0 = RTO scope/measure mode (loop the code forever) bit 1 = RTO is looped back to RTI (externally) bit 4 = sets the SuperIO port to 115,000 baud bit 5 = sets the SuperIO port to 56,000 baud bit 6 = sets the SuperIO port to 38,400 baud bit 7 = sets the SuperIO port to 19,200 baud bit 8 = sets the SuperIO port to 9600 baud
-N <*value*>	Selects the NVRAM testing mode 0 = limited testing 1 = maximum testing 2 = pattern all NVRAM 3 = reads/checks all NVRAM locations 4 = reads/checks limited areas of NVRAM 0x*XXX*0*YYY*£ = dumps NVRAM **Caution:** Do not stop the test while it is testing the NVRAM.
-F <*value*>	Turns FireWire specific operations on or off 0 = no FireWire specific parameters 1 = dumps the physical registers from pages 0, 1, and 7
-U <*value*>	Turns USB-specific operations on or off 0 = no USB-specific parameters 1 = attempts to detect an external USB device and then stops the test

4.5.1 io7bt Troubleshooting Tips

Table 4-6 io7bt Troubleshooting Tips

Problem	Solution
Test failed	Check all connectors and cables used for the hardware that is being tested
Test failed after the connectors were checked	Power cycle the system and test again
Test failed after the system was power cycled	Replace the I brick

4.6 ndir (Directory Memory Test)

The `ndir` test verifies the directory memory on each node. SGI recommends that you run this test for at least 15 passes (`run ndir -e 15`).

Table 4-7 ndir Test Sections

Section	Description
0	Performs a directory memory quick-screen test
1	Performs a directory memory integrity test
2	Performs a directory memory SECDED verification test
3	Performs a directory memory random address/data test
4	Performs a directory memory state verification test

`ndir [-n <nodes>] [-c <cpus>] [-S<section_select>] [-s <start pass>] [-e <end pass>] [-l] [-C] [-b <banks>] [-m <start address>] [-M <end address>] [-?|-h|-H]`

Table 4-8 ndir Command-line Options

Option	Description
`-n` *<nodes>*	Tests the specified node (default: all available nodes)
`-c` *<cpus>*	Uses the specified CPUs (default: all available CPUs)
`-S` *<section_select>*	Runs the specified test sections (Refer to Table 4-7 for valid `ndir` test sections; default: `0x1F` = all sections.)
`-s` *<start pass>*	Specifies the starting pass count (default: `0`)
`-e` *<end pass>*	Specifies the ending pass count (default: `1`)
`-l`	Loops forever between the starting and ending pass
`-C`	Selects continue-on-error mode
`-b` *<banks>*	Uses the specified banks
`-m` *<start address>*	Selects the starting memory address
`-M` *<end address>*	Selects the ending memory address
`-h|-H|-?`	Displays help information

4.6.1 ndir Troubleshooting Tips

If an error occurs, check the error log (the `el` command) for additional information. To decode information in the error log, use the sMDK Error Log Decode Tool Web site:

`http://wwwcf.americas.sgi.com/PUBLIC/sn1_diags/elog_decode/elog_decode.cgi`

4.7 netkiller (Gigabyte System Network Test)

The `netkiller` test is a stress test that verifies GSN XIO boards and X bricks. SGI recommends that you run this test for at least 200 passes (`run netkiller -t st_loopback -i 1 -e 200 -x 1 -v 1`).

`netkiller [-t <testname>] [-e <num>] [-h <num>] [-i<num>] [-p <num>] [-v <num>] [-x <num>] [-o]`

Table 4-9 netkiller Command-line Options

Option	Description
`-t <testname>`	Runs the specified test (`xtwalkingbus`, `shacreg`, `shac_reset`, `fwpreset`, `ssram_addr`, `ssram_switching`, `ssram_randstress`, `st_loopback`, `admin_check`)
`-e <num>`	Specifies the ending pass count (default: 800)
`-h <num>`	Specifies the hop count
`-i <num>`	Enables internal loopback for the GSN board 0 = disables anything else = enables (default)
`-p <num>`	Disables SSRAM parity 0 = enables anything else = disables
`-v <num>`	Enables verbose mode 0 = disables anything else = enables
`-x <num>`	Runs the specified test case 1 = 1-way striping, 16-M bufx, LLP pattern, 16-M fixed length, data check on, multiple VC 2 = 1-way striping, 16-M bufx, random pattern, random length generated every pass, data check on, multiple VC 3 = 8-way striping, 2-K bufx, random pattern, 16-M fixed length every pass, data check on, mixed st and raw 4 = 1-way striping, 16-M bufx, random pattern, 50 x 2-K random length data check on (default) 5 = 8-way striping, 16-M bufx, LLP pattern, 16-M fixed length data check off
`-o`	Ignores unused ports

4.8 nmem (Node Memory Test)

The nmem test verifies that the memory on each node is operating properly. SGI recommends that you run this test for at least 15 passes (`run nmem -e 15`).

Table 4-10 nmem Test Sections

Section	Description
0	Performs a memory quick-screen test
1	Performs a memory integrity test
2	Performs a memory SECDED verification test
3	Performs a memory random address/data test
4	Performs a memory fetch-and-op test

`nmem [-n <nodes>] [-c <cpus>] [-S <section_select>] [-s <start pass>] [-e <end pass>] [-l] [-C] [-b <banks>] [-m <start address>] [-M <end address>] [-?|-h|-H]`

Table 4-11 nmem Command-line Options

Option	Description
`-n` *<nodes>*	Tests the specified node (default: all available nodes)
`-c` *<cpus>*	Uses the specified CPUs (default: all available CPUs)
`-S` *<section_select>*	Runs the specified test sections (Refer to Table 4-10 for valid nmem test sections; default: `0x1F` = all sections.)
`-s` *<start pass>*	Specifies the starting pass count (default: `0`)
`-e` *<end pass>*	Specifies the ending pass count (default: `1`)
`-l`	Loops forever between the starting and ending pass
`-C`	Selects continue-on-error mode
`-b` *<banks>*	Uses the specified memory banks
`-m` *<start address>*	Selects the node memory starting address
`-M` *<end address>*	Selects the node memory ending address
`-h|-H|-?`	Displays help information

4.8.1 nmem Troubleshooting Tips

If an error occurs, check the error log (the `el` command) for additional information. To decode information in the error log, use the sMDK Error Log Decode Tool Web site:

`http://wwwcf.americas.sgi.com/PUBLIC/sn1_diags/elog_decode/elog_decode.cgi`

4.9 rtrb (Router Basic Test)

The `rtrb` test checks the router ASIC local block and lookup table register integrity and basic functionality. SGI recommends that you run this test for at least 40 passes (`run rtrb -e 40`).

Table 4-12 rtrb Test Sections

Section	Description
0	Performs a router local block register integrity test
1	Performs a scratch register functional test
2	Performs a vector routing test

`rtrb [-s <starting pass>] [-e <ending pass>] [-S <section select>] [-h|-H|-?]`

Table 4-13 rtrb Command-line Options

Option	Description
`-s <starting pass>`	Specifies the starting pass count
`-e <ending pass>`	Specifies the ending pass count (default: run forever)
`-S <section select>`	Runs the specified test sections (hexadecimal bitmask; refer to Table 4-12 for valid `rtrb` test sections)
`-h|-H|-?`	Displays help information

4.9.1 rtrb Troubleshooting Tips

It is normal for UART timeouts to occur while running section 0 of `rtrb`.

4.10 rtrdmp (Router Dump Utility)

The `rtrdmp` utility dumps and displays various types of router registers from any router ASIC in a system.

`rtrdmp [-t <register type>] [-r <router number>] [-p <port select>] [-h|-H|-?]`

Table 4-14 rtrdmp Command-line Options

Option	Description
`-t <register type>`	Dumps the specified type of register `b` = barrier `L` = local block `p` = port `l` = local LUT `g` = global LUP
`-r <router number>`	Dumps and displays the specified router ASIC (default: `0`)
`-p <port select>`	Uses the specified port (default: `1`)
`-h|-H|-?`	Displays help information

4.11 rtrreg (Router Register Read/Write Utility)

The `rtrreg` utility reads from or writes to any router register in a system.

`rtrreg [-f <function>] [-a <register address>] [-d <register data>] [-r <router number>] [-h|-H|-?]`

Table 4-15 rtrreg Command-line Options

Option	Description
`-f <function>`	Performs the specified operation `r` = reads (default) `w` = writes
`-a <register address>`	Uses the specified router register address (default: `0`)
`-d <register data>`	Uses the specified data for write operations (default: `0`)
`-r <router number>`	Reads from or writes to the specified router ASIC `0xff` = all router ASICs (default: `0`)
`-h|-H|-?`	Displays help information

4.12 rtrt (Router Traffic Test)

The `rtrt` test checks the router network, router ASIC, hub ASIC, and the interconnecting cables. SGI recommends that you run this test for at least 100 passes (`run rtrt -e 100`).

Table 4-16 rtrt Test Sections

Section	Description
0	Performs a one-to-one traffic test
1	Performs an all-to-all traffic test
2	Performs an all-to-one traffic test
3	Performs a random traffic test

`rtrt [-s` *<starting pass>*`] [-e` *<ending pass>*`] [-S` *<section select>*`] [-c` *<virtual cpu select>*`] [-n` *<virtual node select>*`] [-D <0|1>] [-B <0|1>] [-b <0|1>] [-v <0|1>] [-t` *<sync type>*`] [-d` *<data types>*`] [<-d 0x80> -u` *<user data>*`] [-h|-H|-?]`

Table 4-17 rtrt Command-line Options

Option	Description
`-s` *<starting pass>*	Specifies the starting pass count (default: `0x0`)
`-e` *<ending pass>*	Specifies the ending pass count (default: `0` = runs forever)
`-S` *<section select>*	Runs the specified test sections (hexadecimal bitmask; refer to Table 4-16 for valid `rtrt` test sections; default: `0xf`)
`-c` *<virtual cpu select>*	Uses the specified CPUs (use commas for lists and dashes for ranges; default: all)
`-n` *<virtual node select>*	Uses the specified virtual nodes (use commas for lists and dashes for ranges; default: all)
`-D <0|1>`	Turns data checking on or off `0` = off `1` = on (default)
`-B <0|1>`	Turns background barrier enable on or off `0` = off (default) `1` = on
`-b <0|1>`	Turns background hub scratch register packets enable on or off `0` = off (default) `1` = on
`-v <0|1>`	Turns background vector scratch register packets enable on or off `0` = off (default) `1` = on

Table 4-17 (continued) rtrt Command-line Options

Option	Description
`-t` *<sync type>*	Uses the specified synchronization method `f` = fetch and increment (default) `b` = barrier
`-d` *<data types>*	Selects the data types to run (hexadecimal bitmask) 0x1 = alternating 0x2 = sliding 0's and 1's 0x4 = checkerboard 0x8 = address 0x10 = negative address 0x20 = random 0x40 = simultaneous switching 0x80 = user data pattern (default: `0x7f`)
`<-d 0x80> -u` *<user data>*	Uses the specified user data pattern with the `-d` option
`-h|-H|-?`	Displays help information

4.13 scct (Cache Coherency Test)

The `scct` test is a system-level test that tests global cache coherence and flow control mechanisms by *false-sharing* sets of cache lines. SGI recommends that you run this test for at least 100 passes (`run scct -e 100`).

Table 4-18 scct Test Sections

Section	Description
0	Performs a basic addressing test
1	Performs an all-to-one victims test
2	Performs an all-to-one interventions test
3	Performs an all-to-all test

`scct [-h|-H|-?] [-s <start pass>] [-e <end pass>] [-l] [-S <bitmask>] [-c <cpu list>] [-m <value>] [-M <value>] [-b <select mask>] [-g <group mode>] [<-g 3|4> -G <group size>] [-v <verbosity level>]`

Table 4-19 scct Command-line Options

Option	Description
`-h|-H|-?`	Displays help information
`-s <start pass>`	Specifies the starting pass count (default: 0)
`-e <end pass>`	Specifies the ending pass count
`-l`	Loops between the starting and ending pass
`-S <bitmask>`	Runs the specified test sections (bitmask; refer to Table 4-18 for valid `scct` test sections)
`-c <cpu list>`	Uses the specified virtual CPUs
`-m <value>`	Specifies the lowest physical byte address that can be accessed in each node
`-M <value>`	Specifies the highest physical byte address that can be accessed in each node
`-b <select mask>`	Uses the specified physical memory banks in each node (bitmask) bit 0 = bank 0 bit 1 = bank 1 ... bit 7 = bank 7

Table 4-19 (continued) scct Command-line Options

Option	Description
`-g` *<group mode>*	Selects the CPU group mode 0 = CPUs share the same PI 1 = one CPU per PI within a node 2 = all available CPUs within a node 3 = random selection made before the first pass 4 = random selection made before each pass
`<-g 3|4> -G` *<group size>*	Specifies a target group size (1-8; 0 = randomly selects a size between 2 and 5)
`-v` *<verbosity level>*	Selects the verbosity level of the test (0-2)

4.14 System_Level_Test (System-level Stress Test)

The `System_Level_Test` simulates customer use of the hardware by generating data traffic through the hub ASIC and the rest of the system. SGI recommends that you run this test for at least 800 passes (`run System_Level_Test -e 800`).

`System_Level_Test [-h] [-s <start pass>] [-e <end pass>] [-l] [-u <value>] [-t <test>] [-r <register>] [-f <field>] [-i <value>] [-j <value>] [-n <node>] [-c <CPUs>] [-d <option>] [-v <verbose messages>] [-w no_trace] [-o]`

Table 4-20 System_Level_Test Command-line Options

Option	Description
`-h`	Displays help information
`-s` *<start pass>*	Specifies the starting pass count (default: `1`)
`-e` *<end pass>*	Specifies the ending pass count (default: `100000000`)
`-l`	Loops between the starting and ending pass
`-u` *<value>*	Specifies how often (in passes) the test should update the user (default: `10`)
`-t` *<test>*	Runs the specified tests `BTE` = BTE test `Memory_CC` = random memory/cache coherency test `IO` = GSN test `PIO` = PIO interface test `CPU` = random instruction test `All` = `BTE`, `Memory_CC`, `IO`, and `PIO` tests (default)
`-r` *<register>*	Modifies the specified hub registers
`-f` *<field>*	Modifies the specified hub field
`-i` *<value>*	Specifies the hub register field minimum value
`-j` *<value>*	Specifies the hub register field maximum value
`-n` *<node>*	Uses the specified nodes (default: all)
`-c` *<CPUs>*	Uses the specified CPUs (default: all)

Table 4-20 (continued) System_Level_Test Command-line Options

Option	Description
`-d <option>`	Dumps and displays the specified values `dump_all` = displays addresses, register, instructions `dump_addrs` = displays code and data addresses `dump_gpr_fpr` = displays initial register values `dump_instrs` = displays randomly generated instructions `dump_HUB_Reg_default` = displays hub default register values `dump_HUB_Reg_modified` = displays hub modified register values `dump_BTE_parms` = displays BTE interface table `dump_BTE_addrs` = displays BTE saved addresses `dump_BTE_init_regs` = displays BTE initial values `dump_BTE_final_regs` = displays BTE final values `dump_BTE_results` = displays BTE destination and notification results `dump_BTE_initiate_msg` = displays BTE initiate messages (default: no dump)
`-v <verbose messages>`	Enables or disables the specified verbose messages `pass_info` = displays detailed diagnostic information every pass `ErrLog_info` = does not display error log messages (default: no verbose messages)
`-w no_trace`	Does not write the trace table entries (default: writes trace table entries)
`-o`	Runs the test in I/O loopback mode (default: external loopback mode)

4.14.1 System_Level_Test Troubleshooting Tips

Table 4-21 System_Level_Test Troubleshooting Tips

Problem	Solution
Test failed	Loop on the failing pass
Test failed	Loop on a set of passes surrounding the failing pass
Test failed	Print the generated instructions, addresses, and data
Test failed	View the trace table

4.15 topology (Display Topology Utility)

The `topology` utility displays the network topology of the system.

4.16 xbg (Xbridge Test)

The `xbg` test checks the functionality of the Xbridge ASIC in the P and I bricks. SGI recommends that you run this test for at least 10 passes (`run xbg -e 10`).

Table 4-22 xbg Test Sections

Section	Description
0	Performs a basic register test
1	Performs an ROA test
2	Performs an interrupt test
3	Tests PCI configuration and control
4	Performs a PCI DMA basic test
5	Verifies PCI page-mapped addressing
6	Verifies PCI direct-mapped addressing
7	Verifies PCI DMA byte-swap functionality
8	Verifies PCI DMA buffers
9	Performs a PCI DMA random test
10	Performs a PCI DMA concurrent test

`xbg [-n <vnode>] [-i <io_brick>] [-S <bit_mask>] [-p <port_select>] [-d <device_select>] [-C <continue_flag>] [-s <start pass>] [-e <end_pass>] [-l <loop_count>] [-q] [-h|-H]`

Table 4-23 xbg Command-line Options

Option	Description
`-n <vnode>`	Runs the diagnostic on the virtual node with the specified virtual node number (default: `0xffffffffffffffff` = all available nodes)
`-i <io_brick>`	Tests the virtual I/O brick with the specified virtual I/O brick number (default: `0xffffffffffffffff` = all available I/O bricks)
`-S <bit_mask>`	Runs the specified test sections (bitmask; refer to Table 4-22 for valid xbg test sections) bit 0 = section 0 bit 1 = section 1 ... bit 10 = section 10 `0x00000000000007ff` = all sections (default)

Table 4-23 (continued) xbg Command-line Options

Option	Description
`-p` *<port_select>*	Uses the specified port (bitmask) bit 8 = XIO port 8 bit 9 = XIO port 9 bit 10 = XIO port A ... bit 15 = XIO port F `0x000000000000f300` = all XIO ports (default)
`-d` *<device_select>*	Tests the specified device bit 1 = port 8, device 1 bit 2 = port 8, device 2 ... bit 8 = port 8, device 8 bit 9–16 = port 9, devices 1-8 bit 17–24 = port A, devices 1-8 bit 25–32 = port B, devices 1-8 bit 33–40 = port C, devices 1-8 bit 41–48 = port D, devices 1-8 bit 49–56 = port E, devices 1-8 bit 57–64 = port F, devices 1-8 `0xffffffffffffffff` = all devices (default)
`-C` *<continue_flag>*	Performs the specified action when an error occurs 0 = continues on error 1 = stops (default) 2 = fills buffer and stops testing
`-s` *<start pass>*	Specifies the starting pass count (default: `0x0000000000000000`)
`-e` *<end_pass>*	Specifies the ending pass count 0 = runs forever (default: `0x0000000000000002`)
`-l` *<loop_count>*	Loops the specified number of times from `-s` to `-e` (default: `0x0000000000000001` = loop mode off)
`-q`	Turns on quick-look mode (default: off)
`-h` | `-H`	Displays help information

Chapter 5

Online Diagnostics

Online diagnostics verify system hardware while the operating system is running.

5.1 Common Online Diagnostic Options

Table 5-1 Common Online Diagnostic Command-line Options

Option	Description
`-h|-help`	Displays help information
`--<test_name>`	Prevents the specified test from running
`-all`	Runs all standard tests
`-color`	Colorizes pass/fail status
`-nocolor`	Does not colorize pass/fail status
`-config <filename>`	Loads the specified configuration file
`-c|-cont|-continue`	Ignores errors and continues testing
`-forever`	Runs the diagnostic indefinitely
`-hwdebug <level>`	Selects the verbosity of hardware debugging information (0-5; default: 0)
`-interact`	Allows the test to run interactively
`-noESP`	Disables logging of diagnostic events to ESP
`-nonstd`	Runs all nonstandard tests
`-nonrequired`	Does not automatically select tests that are required by other tests being run
`-operator`	Provides minimal output to the screen
`-runtime <time>`	Runs the test for the specified time (in minutes)
`<test_name>=<number>`	Runs the specified test for the specified number of times
`HWDEBUG=<level>`	Same as `-hwdebug`
`INDENT_STEP=<step>`	Indents the text by the specified number of spaces (default: 2)
`LOG=<filename>`	Copies diagnostic output to the specified file
`MAXERR=<number>`	Exits after the specified number of tests have failed (default: 1)

Table 5-1 (continued) Common Online Diagnostic Command-line Options

Option	Description
MAX_ERRORS=<*number*>	Exits after the specified number of errors have occurred (default: 20)
META=<*number*>	Prints out META information when the specified number of loops have completed
REPEAT=<*loops*>	Runs the list of tests for the specified number of times (default: 1)
WIDTH=<*number*>	Sets the width of the diagnostic messages to the specified number (default: 59)

5.2 Common Online Diagnostic Output Tags

Table 5-2 Common Online Diagnostic Output Tags

Code	Description
ABRT	Explains the error that caused the diagnostic to unexpectedly exit
CMDL	Displays the command-line used to start the diagnostic
CODE	Calls out a specific board or chip
DIAG	Displays information about what may be causing a failure
HDBG	Displays information useful for debugging the hardware
HRTB	Shows that the diagnostic is still running during time-consuming operations
INFO	Displays general information about the hardware or operations of the diagnostic
LOOP	Signals the end of a loop
META	Displays a summary of the diagnostic (green = pass, red = fail, yellow = unresolved)
NOTE	Displays important diagnostic information
REV	Displays the revision level of the diagnostic
RSLT	Displays the result of the most recent test (green = pass, red = fail, yellow = unresolved)
TEST	Signifies the start of a test
TIME	Displays the current time
TOUT	Appears when the diagnostic exits because the maximum runtime was reached
TRCE	Traces code used when a test fails
****ERROR	Signals that a hardware error was detected

5.3 bridgeloc (PCI Bridge Location Utility)

The `bridgeloc` utility displays all PCI bridge vertices in the system and identifies which online diagnostics require which vertex.

5.4 grizzly

The `grizzly` system-level test simulates normal and heavy user loads on the system to stress test the entire system.

Caution: Do not run `grizzly` with other user applications or tests.

```
/usr/diags/bin/<griz_script|grizzly [-iter <1 arg(s)>]
[-numProc <1 arg(s)>] [-regionSize <1 arg(s)>]
[-numRegion <1 arg(s)>] [-numFiles <1 arg(s)>]>
```

Table 5-3 grizzly Command-line Options

Option	Description
`-iter` *<1 arg(s)>*	Specifies the number of iterations (default: 200)
`-numProc` *<1 arg(s)>*	Starts the specified number of processes or threads (default: all)
`-regionSize` *<1 arg(s)>*	Uses the specified size of the memory region
`-numRegion` *<1 arg(s)>*	Uses the specified number of memory regions
`-numFiles` *<1 arg(s)>*	Uses the specified number of files

5.5 oldisk (Disk Test)

The `oldisk` test checks a system disk by writing and reading data patterns to the disk and comparing the data.

Note: The `oldisk` test will not work over the Network File System (NFS).

```
/usr/diags/bin/oldisk <-filename file> [-h|-help]
[-filesize <size>] [-async-direct] [-async-buffered]
[-sync-direct] [-sync-buffered] [-patterns <patterns>]
[-paranoid]
```

Table 5-4 oldisk Command-line Options

Option	Description
`-filename` *<file>*	Uses the specified file for testing
`-h|-help`	Runs with an interactive help menu
`-filesize` *<size>*	Uses the specified amount of disk space (in MB; default: 128)
`-async-direct`	Accesses the drive asynchronously by bypassing the internal buffer of the operating system (default)
`-async-buffered`	Accesses the drive asynchronously by using the internal buffer of the operating system
`-sync-direct`	Accesses the drive synchronously by bypassing the internal buffer of the operating system
`-sync-buffered`	Accesses the drive synchronously by using the internal buffer of the operating system
`-patterns` *<patterns>*	Uses the specified data patterns (`zeros`, `ones`, `checker`, `quadword`, `halfword`, `counting`, and `random`; separate lists with commas; default: all)
`-paranoid`	Double-checks every data buffer as it is filled. **Note:** This option prevents a memory error from appearing as a disk error, but slightly increases execution time.

5.5.1 oldisk Troubleshooting Tips

If a memory error shows up as a disk error, use the `-paranoid` option.

5.6 olenet (Ethernet Test)

The `olenet` test verifies the I-brick-based Ethernet functionality.

Caution: Do not run `olenet` with a user job that uses the I-brick-based Ethernet port.

```
/usr/diags/bin/olenet [-enet-info] [-enet-probe]
[-loop-ext-10|-loop-ext-100|-loop-int-ioc3|-loop-int-phy
<INTERFACE=interface> [PACKETS=<packets>]]
<-v|-vertex pci-bridge-vertex>
```

Table 5-5 olenet Command-line Options

Option	Description
`-enet-info`	Displays information on the current state of the Ethernet controller (default)
`-enet-probe`	Performs a simple register test of the Ethernet controller
`-loop-ext-10 <INTERFACE=interface> [PACKETS=packets]`	Performs an external loopback test at 10 Mbps
`-loop-ext-100 <INTERFACE=interface> [PACKETS=packets]`	Performs an external loopback test at 100 Mbps
`-loop-int-ioc3 <INTERFACE=interface> [PACKETS=packets]`	Performs an internal loopback test on the IOC3 chip
`-loop-int-phy <INTERFACE=interface> [PACKETS=packets]`	Performs an internal loopback test on the PHY chip
`INTERFACE=<interface>`	Uses the specified interface for the loopback test
`PACKETS=<packets>`	Sends the specified number of packets in a loopback test (default: `1000`)
`-v|-vertex <pci-bridge-vertex>`	Specifies the hardware graph vertex of the Ethernet controller **Note:** Use the `bridgeloc` utility to find the *pci-bridge-vertex*.

5.7 olmem (Memory Test)

The `olmem` test verifies the memory and cache components in C bricks.

Caution: Do not run `olmem` with user jobs that have CPU affinity.

```
/usr/diags/bin/olmem [-addr-pattern] [-memaddr] [-tagram]
[MEM=<size>]
```

Table 5-6 olmem Command-line Options

Option	Description
`-addr-pattern`	Runs the address pattern test
`-memaddr`	Runs the moving inversion memory test
`-tagram`	Runs a quick test that exercises the cache
`MEM=<size>`	Tests the specified amount of memory (in MB; default: 80% of free memory)

5.7.1 olmem Troubleshooting Tips

If `olmem` exits with the following message, check the specified path for memory errors:

```
ABRT BUS ERROR: This may be due to a double bit error. Check
ABRT /var/adm/SYSLOG
```

5.8 olpci (PCI Bridge Dump Utility)

The `olpci` utility lists the online diagnostics that are applicable to each vertex of the given PCI bridge.

```
/usr/diags/bin/olpci [-pciDiagList|--pciDiagList]
[-pciCfgSpace|--pciCfgSpace] <-v|-vertex pci-bridge-vertex>
```

Table 5-7 olpci Command-line Options

Option	Description
`-pciDiagList`	Lists online diagnostics for each hardware graph vertex (default)
`--pciDiagList`	Does not list the online diagnostics for each graph vertex
`-pciCfgSpace`	Displays the configuration registers and attached devices
`--pciCfgSpace`	Does not display the configuration registers and attached devices
`-v|-vertex <pci-bridge-vertex>`	Specifies the hardware graph vertex of the PCI bridge **Note:** Use the `bridgeloc` utility to find the *pci-bridge-vertex*.

5.9 olperi (Processor Element Random Instruction Test)

The `olperi` test checks the floating-point, integer, branch, and memory load/store operations of the processor chip user mode.

Caution: If `olperi` runs with user jobs, ensure that you limit the number of CPUs being tested.

`/usr/diags/bin/olperi [-r <processor list>] [-f] [-a] [-b] [-m] [-s <starting seed index>] [-i <number of instructions>] [-q <quick mode>]`

Table 5-8 olperi Command-line Options

Option	Description
`-r` *<processor list>*	Runs the test on the processors with the specified processor numbers (Use commas for lists and the word `to` for ranges; default: all.)
`-f`	Includes the floating-point instruction set in the test vector (default: all instruction sets)
`-a`	Includes the integer instruction set in the test vector (default: all instruction sets)
`-b`	Includes the branch instruction set in the test vector (default: all instruction sets)
`-m`	Includes the memory load/store instruction set in the test vector (default: all instruction sets)
`-s` *<starting seed index>*	Generates the test vector with the indicated seed index
`-i` *<number of instructions>*	Specifies the number of machine instructions in each test vector (`16-1024`)
`-q` *<quick mode>*	Compares a checksum that represents the result of all the passes (default: not quick mode) **Note:** A decrease in execution time is evident only when running with a large number of CPUs or for long periods of time.

5.9.1 olperi Troubleshooting Tips

To further isolate a problem, rerun `olperi` using the `-i` option to reduce the number of instructions to test.

5.10 olrti (Real-time Interrupt Test)

The `olrti` test checks the capability of the real-time interrupt to send and receive interrupts. It tests the system by sending interrupts from the RTO connector to the RTI connector.

Caution: Do not run with user jobs that use the RTI ports under test.

`/usr/diags/bin/olrti [-internal] [-external <-v|-vertex` *EI-vertex*`>] [-on|-off [LENGTH=<`*time*`>]] [-pulse|-square <PERIOD=`*time*`|FREQ=`*frequency*`> [LENGTH=<`*time*`>]] [-strobe] [-receive <TIMEOUT=`*time*`>]`

Table 5-9 olrti Command-line Options

Option	Description
`-v|-vertex <`*EI-Vertex*`>`	Specifies the hardware graph vertex of the external interrupts
`-internal`	Tests only the internal loopback capability (default: internal and external loopback tests)
`-external <-v|-vertex` *EI-Vertex*`>`	Tests only the external loopback capability (default: internal and external loopback tests)
`-on [LENGTH=`*time*`]`	Asserts the RTO connector to High
`-off [LENGTH=`*time*`]`	Asserts the RTO connector to Low (default)
`-pulse <PERIOD=`*time* `|FREQ=`*frequency*`> [LENGTH=`*time*`]`	Outputs a pulse train
`-square <PERIOD=`*time* `|FREQ=`*frequency*`> [LENGTH=`*time*`]`	Outputs a square wave
`-strobe`	Outputs a single interrupt
`-receive <TIMEOUT=`*time*`>`	Monitors for interrupts until a time-out occurs
`PERIOD=<`*time*`>`	Specifies the period of time (in microseconds) between pulses or the width of a square wave
`FREQ=<`*frequency*`>`	Specifies the frequency (in Hertz) of a pulse train and a square wave
`LENGTH=<`*time*`>`	Specifies the length (in seconds) of a pulse train, a square wave, and the output for the `-on` and `-off` options
`TIMEOUT=<`*time*`>`	Specifies the amount of time (in seconds) that may pass between interrupts in receive mode

5.11 olrtr (Router Test)

The `olrtr` test verifies the router components in the R bricks and tests network data paths, network addressing, and network-level cache coherency.

Caution: Do not run `olrtr` with user jobs; false failures could occur.

`/usr/diags/bin/olrtr [-P <file>] [-q <file>] [-Q] [-r <file>] [-R] [-G <component name>]`

Table 5-10 olrtr Command-line Options

Option	Description
`-P` *<file>*	Tests a set of network paths in the specified file
`-q` *<file>*	Tests the nodes in the specified file
`-Q`	Tests all the nodes in the system **Note:** Ensure that no one else is using the system when you use this option.
`-r` *<file>*	Randomly tests the nodes in the specified file
`-R`	Randomly tests the nodes in the system (default) **Note:** Ensure that no one else is using the system when you use this option.
`-G` *<component name>*	Lists paths needed to test all of the ports of a router ASIC or MetaRouter

5.11.1 olrtr Troubleshooting Tips

Table 5-11 olrtr Troubleshooting Tips

Problem	Solution
Link faults occurred	Use `linkstat(1)` to view the faults
Test stalled	Check the CPU time of processes by using `ps -af | grep olrtr`
Test processes are not getting CPU time	Kill the test and wait until the target nodes are available
Test failed while using `linkstat -ac`	Any activity that is on the system during `linkstat -ac` can cause inaccurate results
Stale or corrupted data was not detected while using the `-P` option	Use the `-r`, `-q`, `-R`, or `-Q` options instead

5.12 olsio (SuperIO Port Test)

The `olsio` test runs internal loopback tests on Port A and B (if port B exists) of an SuperIO (SIO) port using the PIO and DMA transfers.

Caution: Do not run with user jobs that use the SIO ports under test.

```
/usr/diags/bin/olsio <-v|-vertex ioc3-vertex> [-Internal|-I]
[-SinglePortExternal|-SPExt] [-DualPortExternal|-DPExt
[-flowcontrol|-f <-rs232>]] [-passes|-p <number-of-passes>]
[-rs232] [-rs422] [-A] [-B] [-PIO] [-DMA]
[-baud|-b baud-rate]
```

Table 5-12 olsio Command-line Options

Option	Description
`-v|-vertex` *<ioc3-vertex>*	Specifies the hardware graph vertex of the SIO port **Note:** Use the `bridgeloc` utility to find the *ioc3-vertex*.
`-Internal|-I`	Runs the internal loopback tests (default)
`-SinglePortExternal|-SPExt`	Runs the external loopback tests on a single port **Note:** Use a loopback plug on the port to be tested.
`-DualPortExternal|-DPExt [-flowcontrol|-f <-rs232>]`	Runs the external loopback tests between ports A and B of the same SIO card **Note:** Use a loopback cable between the ports to be tested.
`-passes|-p` *<number-of-passes>*	Runs the test for the specified number of passes (default: 10)
`-rs232`	Selects only RS-232 mode (default: RS-232 and RS-422 modes)
`-rs422`	Selects only RS-422 mode (default: RS-232 and RS-422 modes)
`-A`	Runs on Port A only (default: Port A and B—if Port B exists)
`-B`	Runs on Port B only (default: Port A and B—if Port B exists)
`-PIO`	Uses only PIO transfers (default: PIO and DMA transfers)
`-DMA`	Uses only DMA transfers (default: PIO and DMA transfers)
`-flowcontrol|-f`	Turns on flow control for the PIO handshaking test that is part of the dual port external loopback tests under RS-232 mode

Table 5-12 (continued) olsio Command-line Options

Option	Description
`-baud|-b` *baud-rate*	Sets the desired baud rate (default: 460 KB/s)

5.12.1 olsio Troubleshooting Tips

If an external loopback test fails in RS-422 mode (the `-rs422` option) while performing DMA transfers, it does not indicate faulty hardware.

5.13 olusb (USB Port Test)

The `olusb` test verifies a USB host controller and optionally checks the devices that are attached to that controller.

```
/usr/diags/bin/olusb [-h|-help] [-usb-hc-check]
[-usb-inv-probe [SAVE=<filename>] [COMPARE=<filename>]]
<-v|-vertex vertex>
```

Table 5-13 olusb Command-line Options

Option	Description
`-h|-help`	Runs with an interactive help menu
`-usb-hc-check`	Runs the host controller check test (default) **Note:** The USB bus is not available to the system while this test is running, but is available once the test is finished.

`-usb-inv-probe`
`[SAVE=`*filename*`]`
`[COMPARE=`*filename*`]`
	Runs the inventory probe test
	Note: The USB bus is not available to the system while this test is running and it may not be available when the test is complete. (This would affect any keyboards or mice that are attached via USB.)

`SAVE=<`*filename*`>` `[COMPARE=`*filename*`]`	Saves the contents and state of the bus found in the inventory probe test to the specified file. **Note:** This file can be used at a later run for comparison.
`COMPARE=<`*filename*`>`	Compares the current state and topology of the bus found in the inventory probe test with the state saved in the specified file **Note:** Any differences will be flagged as an error.
`-v|-vertex <`*vertex*`>`	Specifies the hardware graph vertex of the USB controller **Note:** Use the `bridgeloc` utility to find the *vertex*.

5.13.1 olusb Troubleshooting Tips

You may experience problems with keyboards or mice that are attached via the USB after running `olusb` with the `-usb-inv-probe` option.

5.14 olvht (HIPPI Interface Test)

The `olvht` test exercises the HIPPI networking hardware by sending and receiving data.

Caution: Do not run `olvht` with a user job that is using the HIPPI interface.

`/usr/diags/bin/olvht <-IField|-I` ***I-Field***`> <-u|-ULpid` ***ULP-id value***`> <d|-device` ***device-name***`> [-a|-action <`***action***`>] [-b|-blocksize <`***blocksize-max***`[:`***blocksize-min***`[:`***block-step***`]]|pass>] [-e|-echofile <`***echo-file-name***`>] [-o|-options <options>] [-p|-pattern <`***pattern***`>] [-p|-passes <`***pass-end***`>[:`***pass-start***`[:`***pass-step***`]]] [TIMEOUT=<`***n***`>] [NUMPEADS=<`***n***`>] [-W|-WaitOnRead <`***wait-on-read value***`>]`

Table 5-14 olvht Command-line Options

Option	Description
`-I|-IField <`***I-Field***`>`	Specifies the I-Field value **Caution:** If you are not familiar with the term *I-Field*, use `olvst` instead of this test.
`-u|-ULpid <`***ULP-id value***`>`	Specifies the ULP-id value (`128-255`) **Caution:** If you are not familiar with the term *ULP id*, use `olvst` instead of this test.
`-d|-device <`***device-name***`>`	Performs the I/O operations on the specified HIPPI device
`-a|-action <`***action***`>`	Performs the specified action `rw` = reads and writes data on the same machine (default) `read` = waits for and reads data from the cooperating process `write` = writes data to the specified host **Note:** Ensure that you start a read action before you start a write action
`-b|-blocksize <`***blocksize-max***`[:`***blocksize-min***`[:`***block-step***`]]|pass>`	Uses the specified block size for I/O operations (***blocksize-max***: `1-65534`; default: `16384`)
`-e|-echofile <`***echo-file-name***`>`	Writes the first data buffer to the specified file; no I/O is performed
`-o|-options <`***options***`>`	Turns on the list of options `fast` = does not compare data `verbose` = displays the progress of the test

Table 5-14 (continued) olvht Command-line Options

Option	Description
`-P|-Pattern` *<pattern>*	Uses the specified data pattern `bits` = each byte has a random sequence of consecutive 1-bits `slide0` = first byte: bit 0 is cleared, all other bits are set; rest: circularly left-shifted by 1 bit `slide1` = first byte: bit 0 is set, all other bits are cleared; rest: circularly left-shifted by 1 bit `random` = each byte is set to a random value `all` = all patterns are used, 1 per pass (default) *numeric constant* = used to set each 64-bit word of the data buffer
`-p|-passes` *<pass-end>* [: *pass-start* [: *pass-step*]]	Performs the specified number of passes (*pass-end*: greater than 0; default: `1`) **Note:** *pass-end* can be no more than `65534` when `-b|-blocksize pass` is used
`TIMEOUT=`*<n>*	Specifies the time-out value (default: 10 seconds)
`NUMPEADS=`*<n>*	Specifies the number of reads to post (default: `6`)
`-W|-WaitOnRead` *<wait-on-read value>*	Specifies the number of 0.10-second intervals that the HIPPI driver waits for a read action to occur (`1-254`; `255` = waits forever; default: 0)

5.15 olvst (TCP Socket Test)

The `olvst` test exercises TCP sockets by sending and receiving data.

`/usr/diags/bin/olvst [-a|-action <action>] [-b|-blocksize <blocksize-max[:blocksize-min[:blocksize-step]]|pass>] [-e|-echofile <echo-file-name>] [-H|-Host <host-name>[:port-number] [-o|-options <options>] [-p|-passes <pass-end>[:pass-start[:pass-step]]] [-P|-pattern <pattern>] [TIMEOUT=<n>]`

Table 5-15 olvst Command-line Options

Option	Description
`-a|-action <action>`	Performs the specified action `ping` = elicits ECHO_RESPONSE packets from the host (block size is limited to 16,376 bytes) `rw` = reads and writes data on the same machine (Ensure that you use the `-o|-options verbose` command with this option) `read` = reads data from the cooperating process (Ensure that you use the `-o|-options verbose` command with this option) `write` = writes data to the specified host **Note:** Ensure that you start a `read` action before you start a `write` action.
`-b|-blocksize <blocksize-max[:blocksize-min[:blocksize-step]]|pass>`	Uses the specified block size for I/O operations (*blocksize-max*: greater than 0; default: `4096`)
`-e|-echofile <echo-file-name>`	Writes the first data buffer to the specified file and exits; no I/O is performed
`-H|-Host <host-name>[:port-number]`	Tests the specified host (default: the machine on which the test is running)
`-o|-options <options>`	Turns on the list of options `fast` = does not compare data `verbose` = displays the progress of the test for the rw and read actions
`-p|-passes <pass-end>[:pass-start[:pass-step]]`	Performs the specified number of passes (*pass-end*: greater than 0; default: `1`)

Table 5-15 (continued) olvst Command-line Options

Option	Description
`-P|-pattern <pattern>`	Uses the specified data pattern `bits` = each byte has a sequence of consecutive 1-bits `slide0` = first byte: bit 0 is cleared, all other bits are set; rest: circularly left-shifted by 1 bit `slide1` = first byte: bit 0 is set, all other bits are cleared; rest: circularly left-shifted by 1 bit `random` = each byte is set to a random value `all` = all patterns are used, 1 per pass (default) *numeric constant* = uses the specified number to set each word **Note:** The pattern must be the same on both the receiving and sending hosts.
`TIMEOUT=<n>`	Uses the specified time-out value when asynchronous I/O is performed (default: 10 seconds)

5.16 pandora

The pandora test simulates heavy user loads on the system to stress test processors, memory, I/O devices, network devices, router interconnect hardware, and graphics hardware.

Caution: Do not run pandora with other user applications or tests.

```
/usr/diags/bin/pandora [-v] [-dpca] [-d] [-g <-mount>]
[-mount] [<-runtime 0> -tloops <tloops>]
```

Table 5-16 pandora Command-line Options

Option	Description
-v	Prints pandora version and exits
-dpca	Dumps precomputed array
-d	Does not generate a new sysinfo file
-g <-mount>	Stops after generating sysinfo file
-mount	Mounts optional drives
<-runtime 0> -tloops <*tloops*>	Executes the specified number of test loops (greater than 0)

5.16.1 pandora Troubleshooting Tips

If pandora fails, the error might be the result of one of the following conditions:

- Data miscompares, mount failures, or time-outs might be I/O-related errors.
- Data miscompares might be FPU errors or graphics-related errors.
- Miscompared, lost, duplicate, or damaged packets might be network-related errors.

Table 5-17 pandora Troubleshooting Tips

Problem	Solution
System hung	Perform an NMI and analyze the dump
System crashed	Check the SYSLOG for FRU analysis
Problem occurred with DIMMs	Check the SYSLOG for ECC errors
Problem occurred with router ASICs and links	Check the SYSLOG for excessive check bit or sequencing errors
Problem occurred with router ASICs and links	Use linkstat while running pandora

5.17 torpedo (CPU Instruction Test)

The `torpedo` test performs a floating-point unit (FPU) stress test to test the FPU in each CPU.

Caution: Do not run `torpedo` with other user applications or tests.

```
/usr/diags/bin/torpedo
[-onfailure <abort|cont|loop|repeat|stop>] [-nologseeds]
[<-runtime 0> -rloops <n>] [-ncpus <n>] [-iseed <n>]
[-showallmis] [-tconfig <file>]
```

Table 5-18 torpedo Command-line Options

Option	Description
`-onfailure <abort|cont|loop|repeat|stop>`	Defines how the test should react to failures `abort` = exits immediately without doing any clean-up `cont` = continues testing (default) `loop` = continuously repeats the loop that detected the miscompare `repeat` = repeats the last loop `stop` = stops testing and performs clean-up
`-nologseeds`	Does not dump the random seeds to a file (default: dumps the random seeds to a file)
`<-runtime 0> -rloops <n>`	Executes the specified number of test loops (default: `1000`)
`-ncpus <n>`	Tests the specified number of CPUs (default: the number of usable CPUs found in the system)
`-iseed <n>`	Uses the specified seed as the initial seed value (default: `1`)
`-showallmis`	Prints all miscompares found in a test loop (default: does not show miscompares)
`-tconfig <file>`	Loads the specified configuration file